Advanced Data Structures & Algorithms

An in-depth interview handbook — explained with diagrams

Seven parts · 60+ diagrams · 70+ interview questions with worked answers
Amortized analysis · Segment trees · Union-Find · Graph algorithms
Dynamic programming · String matching · Interview technique

Prepared for **Shikhar Mutta** · September 2026
Companion to the LeetCode practice repository — 935 problems solved
Code samples are C++17, matching the conventions used in that repo.

Contents

PART 1

Complexity & Analysis

The machinery every later part depends on — and the questions that reveal whether you understand it or have memorised it.

Interviewers rarely ask "what is the complexity of quicksort?". They ask something that *looks* like an implementation question and watch whether your analysis is real or recited. This part covers the analysis machinery that the rest of the book leans on.

1.1 Growth rates and why they matter

Every complexity class is a claim about how the running time responds when the input doubles. That framing is more useful in an interview than the formal definition, because it turns into an immediate sanity check: if `n` doubles and your algorithm does four times the work, it is quadratic, whatever the code looks like.

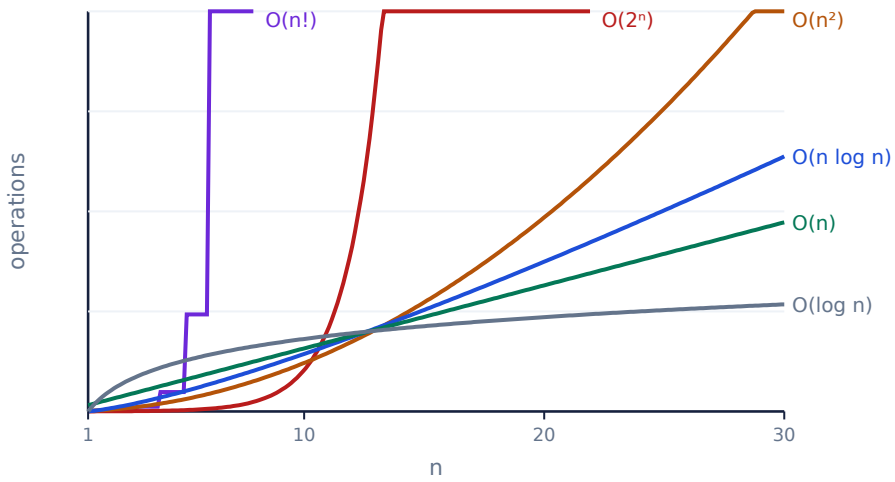

Fig 1.1 — the same axes for every class. The gap between O(n log n) and O(n²) is what most interview optimisations buy you.

Class	n = 10	n = 1,000	n = 10^6	Typical source
O(1)	1	1	1	hash lookup, arithmetic
O(log n)	3	10	20	binary search, balanced tree
O(n)	10	10^3	10^6	single scan, two pointers
O(n log n)	33	10^4	2×10^7	sorting, divide and conquer
O(n²)	100	10^6	10^{12} — too slow	nested loops, all pairs
O(2ⁿ)	1,024	overflow	overflow	subsets, naive recursion
O(n!)	3.6×10^6	overflow	overflow	permutations, brute-force TSP

THE 10^8 RULE

Competitive judges and interviewers both assume roughly **10^8 simple operations per second**. Read the constraints backwards to find the intended complexity: `n ≤ 10^6` means O(n) or O(n log n); `n ≤ 10^5` allows O(n√n); `n ≤ 5000` invites O(n^2); `n ≤ 20` is shouting **bitmask** at you. Saying this out loud at the start of an interview is a strong signal.

1.2 Amortized analysis: three methods

Amortized cost is the average cost per operation *over a worst-case sequence*. It is not average-case analysis: there is no probability involved, and the guarantee holds for every sequence, not for a typical one. This distinction is the single most common thing candidates get wrong, and interviewers ask it precisely because the wrong answer is so easy to give confidently.

The canonical example is `std::vector::push_back`. Most pushes write one element and cost O(1). Occasionally the buffer is full, and the vector allocates a bigger block and copies everything — O(n) for that one push. The worst single push is therefore O(n), yet n pushes cost O(n) in total.

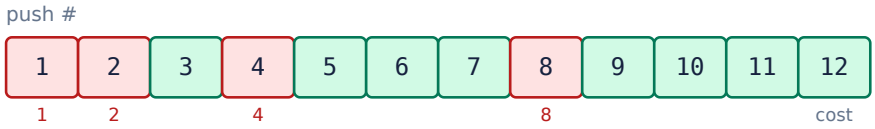

Fig 1.2 — twelve `push_back` calls. Red pushes reallocate and cost 1, 2, 4, 8; every other push costs 1. Total = 12 + 15 = 27 < 3n.

Method 1 — aggregate

Sum the whole sequence and divide. With doubling, the reallocations during n pushes copy 1 + 2 + 4 + … + n/2 < n elements in total. Add the n cheap writes: total < 2n, so the amortized cost is **O(1)**. The geometric series is doing all the work — and it only converges because the growth is *multiplicative*.

WHY GROWING BY A CONSTANT IS A DISASTER

If the vector grew by **+1** each time instead of ×2, every push would reallocate: 1 + 2 + … + n = n(n+1)/2 copies, so **O(n) amortized** per push and O(n^2) to build the array. Interviewers love this follow-up because it checks whether you understand *why* doubling works rather than that it does. Any growth factor > 1 gives O(1); the choice of 2 (or libstdc++'s 2, MSVC's 1.5) is a memory-reuse trade-off, not a complexity one.

Method 2 — accounting (the banker's method)

Charge each operation an invented price. Overcharging cheap operations banks credit that pays for the rare expensive one. Charge **3** per push: 1 pays for writing the new element, 1 is saved to copy that element at the next resize, and 1 is saved to copy one of the older elements that has already spent its own credit. When a resize of size k happens, exactly the k/2 elements added since the last resize have 2 credits each — enough to move all k. Credit never goes negative, so the true cost never exceeds 3 per push.

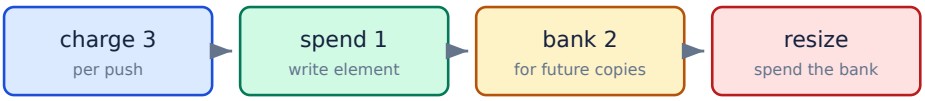

Fig 1.3 — the accounting method: a constant price per operation, with the surplus banked against the rare O(n) event.

Method 3 — potential

Define a potential function Φ on the data structure — stored energy that expensive operations cash in. The amortized cost is the real cost plus the change in potential: `â = c + Φ(after) − Φ(before)`. For the vector, Φ = 2 × (size − capacity/2) works: a cheap push raises Φ by 2 and costs 1, giving 3; a resize costs n but drops Φ by about n, giving O(1). Provided Φ starts at 0 and never goes negative, the sum of amortized costs bounds the sum of real costs.

Method	The idea	Best when
Aggregate	total cost / number of operations	the sequence has obvious structure
Accounting	fixed price per op; bank the surplus	you can point at what pays for what
Potential	Φ measures stored work	several operation types interact

WHERE THIS SHOWS UP

Union-Find (α(n) amortized, §2.3), **monotonic stacks** (each element pushed and popped once, §2.7), **splay trees**, **hash table rehashing**, and the **two-pointer** family — where the inner loop looks nested but each pointer only ever moves forward, so the total is O(n) not O(n²). If you can say *"each element is pushed once and popped at most once, so the total work is linear"*, you have given the aggregate argument correctly.

1.3 The Master Theorem

For a recurrence `T(n) = a·T(n/b) + f(n)` — a subproblems, each of size n/b, plus $f(n)$ to split and combine — compare $f(n)$ against `n^(log_b a)`, the total work in the leaves.

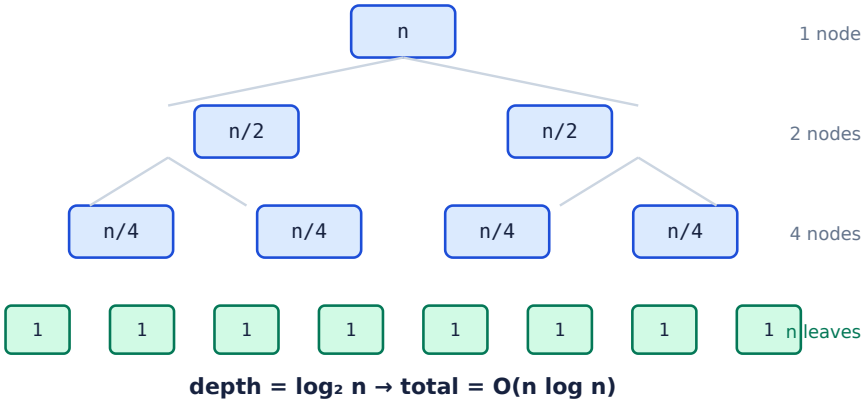

Fig 1.4 — the recursion tree for merge sort, T(n) = 2T(n/2) + O(n). Each level does O(n) work and there are log n levels.

Case	Condition	Result	Example
1 — leaves dominate	f(n) = O(n^(log_b a − ε))	T(n) = Θ(n^(log_b a))	T(n)=8T(n/2)+n² → Θ(n³)
2 — balanced	f(n) = Θ(n^(log_b a))	T(n) = Θ(n^(log_b a) log n)	T(n)=2T(n/2)+n → Θ(n log n)
3 — root dominates	f(n) = Ω(n^(log_b a + ε)) + regularity	T(n) = Θ(f(n))	T(n)=2T(n/2)+n² → Θ(n²)

Recurrence	Solution	Where you meet it
T(n) = T(n/2) + O(1)	Θ(log n)	binary search
T(n) = T(n/2) + O(n)	Θ(n)	quickselect (average), Fig 1.1 partitioning
T(n) = 2T(n/2) + O(1)	Θ(n)	tree traversal, heapify
T(n) = 2T(n/2) + O(n)	Θ(n log n)	merge sort, most divide & conquer
T(n) = 2T(n/2) + O(n log n)	Θ(n log² n)	some CDQ / D&C optimisations
T(n) = T(n−1) + O(1)	Θ(n)	linear recursion
T(n) = T(n−1) + O(n)	Θ(n²)	quicksort worst case, selection sort
T(n) = 2T(n−1) + O(1)	Θ(2ⁿ)	Towers of Hanoi, naive subsets

WHEN THE MASTER THEOREM DOES NOT APPLY

It needs subproblems of *equal* size and an f that is well behaved. It says nothing about `T(n) = T(n/3) + T(2n/3) + n` (unequal split — use a recursion tree; the answer is Θ(n log n)) or about `T(n) = 2T(n/2) + n/log n`, which falls in the gap between cases 1 and 2. Knowing the limits is worth more than reciting the three cases.

1.4 Space complexity and hidden costs

Space is asked about half as often as time and answered badly twice as often, usually because the recursion stack gets forgotten. A recursive function holding no data structures still costs O(depth) space.

Algorithm	Time	Space	The hidden cost
Merge sort	O(n log n)	O(n)	the merge buffer, not the stack
Quicksort	O(n log n) avg	O(log n) avg	stack depth; O(n) if unbalanced
Heap sort	O(n log n)	O(1)	genuinely in-place
DFS on a graph	O(V+E)	O(V)	stack depth — a path graph overflows
BFS on a graph	O(V+E)	O(V)	the frontier can hold most of a level
Recursive Fibonacci	O(2ⁿ)	O(n)	stack depth is only linear
0/1 knapsack (2-D)	O(nW)	O(nW)	rolls to O(W) — §4.2

```cpp
// Two ways to sum a list. Same time, very different space.
long long sum_rec(Node* p) {                 // O(n) stack — overflows near ~10^5 nodes
    return p ? p->val + sum_rec(p->next) : 0;
}
long long sum_itr(Node* p) {                 // O(1) stack
    long long s = 0;
    for (; p; p = p->next) s += p->val;
    return s;
}
```

SAY IT IN THIS ORDER

"Time is O(n log n) — one sort, then a linear scan. Space is O(n) for the auxiliary array, plus O(log n) recursion stack, so O(n) overall. If the input may be modified I can drop that to O(1) auxiliary." Naming the *source* of each term, and offering the trade-off, is what separates a recited answer from an analysed one.

1.5 Interview questions

Q1 What is the difference between amortized and average-case complexity?

Average-case is **probabilistic**: it averages over a distribution of inputs and says nothing about a particular one. Hash table lookup is O(1) average-case — with adversarial keys it is O(n).

Amortized is a **worst-case guarantee over a sequence**. No probability is involved. `push_back` is O(1) amortized: *any* sequence of n pushes costs O(n), and no adversary can make it otherwise.

The clean one-liner: *average-case can be defeated by unlucky input; amortized cannot.* A single amortized-O(1) operation may still take O(n) — which is exactly why amortized bounds are unsuitable for hard real-time systems, where a rehash pausing one insert is unacceptable even though the average is fine.

Q2 std::vector grows by doubling. Why not by a fixed 1000 elements?

Because the total copy cost stops telescoping. With doubling, resizes copy $1 + 2 + 4 + \ldots + n/2 < n$ elements overall — a geometric series bounded by n, so O(1) amortized.

Growing by a constant c means n/c resizes, copying $c + 2c + 3c + \ldots + n = \Theta(n^2/c)$ elements. Amortized cost per push becomes $\Theta(n/c)$ — linear, not constant. The constant only shifts the curve; it stays quadratic.

Any factor strictly greater than 1 gives O(1). The choice between 1.5 and 2 is about memory: with factor 2 the freed blocks can never be coalesced to satisfy the next request ($1+2+4+\ldots+2^{k-1} < 2^k$), whereas 1.5 eventually can reuse them. That is why MSVC uses 1.5 and libstdc++ uses 2.

Q3 Give a worst-case sequence for a hash table with chaining, and fix it.

Insert n keys that all hash to the same bucket. Every lookup then walks a chain of length n: O(n) per operation, $O(n^2)$ to build. If the hash function is public and deterministic — as Java's `String.hashCode` is, and as many language defaults are — an attacker can compute such keys and turn any request handler into a denial-of-service. This is a real, named vulnerability class (*hash flooding*, CVE-2011-4815 and relatives).

Three fixes, in increasing strength:

- **Randomised hashing** — seed the hash with a per-process random value, so the adversary cannot precompute collisions. This is what Python, Rust and modern PHP do.
- **Treeify long chains** — convert a bucket to a balanced BST past a threshold, capping worst case at O(log n). Java 8's `HashMap` does this at 8 entries.
- **Universal / perfect hashing** — pick the hash from a family at random, giving provable expected bounds independent of input.

Q4 T(n) = 2T(n/2) + n/log n. Solve it.

The Master Theorem does **not** apply. Here a = 2, b = 2, so $n^{\log_b a} = n$. We need to compare f(n) = n/log n against n. It is smaller, so case 1 is the candidate — but case 1 requires $f(n) = O(n^{1-\varepsilon})$ for some $\varepsilon > 0$, and n/log n is *not* polynomially smaller than n; it is smaller only by a logarithmic factor. The recurrence falls in the gap.

Expand it instead. At level i there are 2^i subproblems of size $n/2^i$, each costing $(n/2^i)/\log(n/2^i)$, so the level total is $n/(\log n - i)$. Summing $i = 0 \ldots \log n - 1$ gives $n \cdot \Sigma\, 1/(\log n - i) = n \cdot H_{\log n} \approx$ **$\Theta(n \log \log n)$**.

Recognising that the theorem does not apply is the point of the question; the harmonic sum is the follow-up.

Q5 An algorithm is $O(n^2)$. Is it also $O(n^3)$? Is it $\Theta(n^2)$?

Yes to $O(n^3)$. Big-O is an *upper* bound, and upper bounds are not required to be tight: $O(n^2) \subset O(n^3)$. Saying an algorithm is $O(n^3)$ when it is really quadratic is true but useless — which is why interviewers expect the tightest bound you can justify.

Not necessarily to $\Theta(n^2)$. Θ requires matching upper *and* lower bounds. Insertion sort is $O(n^2)$ but not $\Theta(n^2)$, because on already-sorted input it runs in $\Theta(n)$. It *is* $\Theta(n^2)$ in the worst case — and that qualifier is the whole answer. Always state which case you are bounding.

Completeness: Ω is the lower bound, o and ω are the strict versions ($f = o(g)$ means $f/g \to 0$).

Q6 Why is comparison sorting Ω(n log n)?

Model any comparison sort as a binary decision tree: each internal node is one comparison, each leaf is one of the possible output permutations. A correct algorithm must be able to reach every permutation, so the tree has at least $n!$ leaves. A binary tree with L leaves has height $\geq \log_2 L$, and the height is the number of comparisons in the worst case. So the worst case is at least $\log_2(n!)$, which by Stirling's approximation is $n \log_2 n - n \log_2 e + O(\log n) =$ **$\Omega(n \log n)$**.

The bound only constrains algorithms whose *only* operation on keys is comparison. Counting sort, radix sort and bucket sort read the keys' structure instead and run in $O(n + k)$ or $O(d(n + k))$ — they do not violate the bound, they sit outside the model.

Q7 You have O(n log n) and O(n√n) solutions. Which do you ship?

Asymptotically $O(n \log n)$ wins — $\log n$ grows far slower than $\sqrt{n}$, and they cross very early ($\log_2 n < \sqrt{n}$ for all $n \geq 2$). At $n = 10^6$, $\log_2 n \approx 20$ against $\sqrt{n} = 1000$: a 50× gap.

But the honest answer names the constants. A $\sqrt{n}$ algorithm is often square-root decomposition — flat arrays, sequential access, no branching — while an $O(n \log n)$ algorithm may be a pointer-chasing balanced tree with a cache miss per level. For n in the thousands the $\sqrt{n}$ version can genuinely win, and it is usually far shorter to write correctly under time pressure.

What the interviewer wants: *“asymptotically the log version, and that is what I would ship for large n — but I would measure at the real input size, because √n-decomposition has much better constants and I can implement it correctly in five minutes.”*

Q8 What is the complexity of building a heap from n elements, and why is it not O(n log n)?

It is **$O(n)$**. The naive argument — n elements, each sifted down $O(\log n)$ — over-counts, because almost no element sits near the root.

In a heap of n nodes, at height h there are at most $\lceil n/2^{h+1} \rceil$ nodes, and sifting one down costs $O(h)$. The total is $\Sigma_{h=0}^{\log n} (n/2^{h+1}) \cdot O(h) = O(n \cdot \Sigma\, h/2^h)$. That sum converges to 2, so the total is **$O(n)$**.

Half the nodes are leaves and cost nothing; a quarter sift down at most one level. The work is concentrated where the nodes are, which is at the bottom. Note this only holds for the bottom-up `heapify`; inserting n elements one at a time really is $\Theta(n \log n)$.

PART 2

Data Structures in Depth

Not what they do — why they cost what they cost, and which trade-off each one is making.

Interviewers use data structures as a proxy for whether you can reason about trade-offs. Knowing that a heap gives O(log n) insert is table stakes; knowing why `build_heap` is O(n), when a Fenwick tree beats a segment tree, and what path compression actually costs is what the question is for.

2.1 Hash tables: chaining vs open addressing

A hash table maps a key to a bucket index with a hash function, then has to answer one question: what happens when two keys land in the same bucket? The two answers give two structures with genuinely different performance characters.

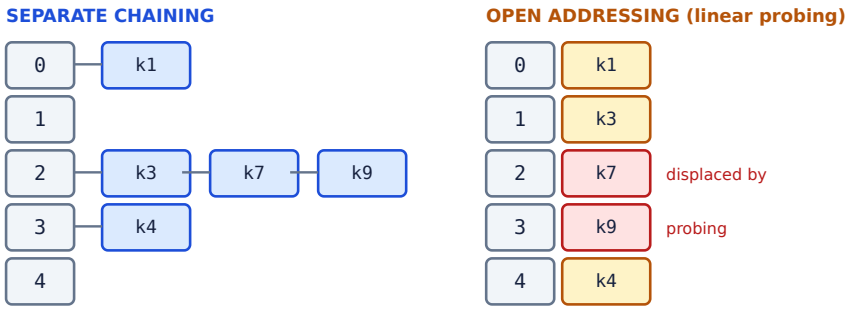

Fig 2.1 — the same five keys, with k3, k7, k9 colliding. Chaining hangs them off the bucket; open addressing pushes them into the next free slots, where they collide with *other* keys in turn.

	Separate chaining	Open addressing
Collision handling	linked list / tree per bucket	probe for another slot
Load factor α	can exceed 1	must stay < 1, degrades badly past 0.7
Failed lookup	1 + α probes	~1/(1−α) probes
Cache behaviour	poor — pointer chasing	excellent — contiguous
Deletion	trivial	needs tombstones or backward shift
Memory	pointer per node	no overhead, but must stay sparse
Used by	`std::unordered_map`, Java `HashMap`	Python `dict`, Google `flat_hash_map`

WHY std::unordered_map IS SLOW

The C++ standard mandates **reference stability** — a reference to an element must stay valid across rehashes — and permits iteration over buckets. Together these force separate chaining with node allocation, so every lookup is a pointer dereference into a random cache line. Open-addressed replacements (`absl::flat_hash_map`, `ankerl::unordered_dense`) are routinely 2–3× faster. If you are asked why a hash map is slower than expected in C++, this is the answer.

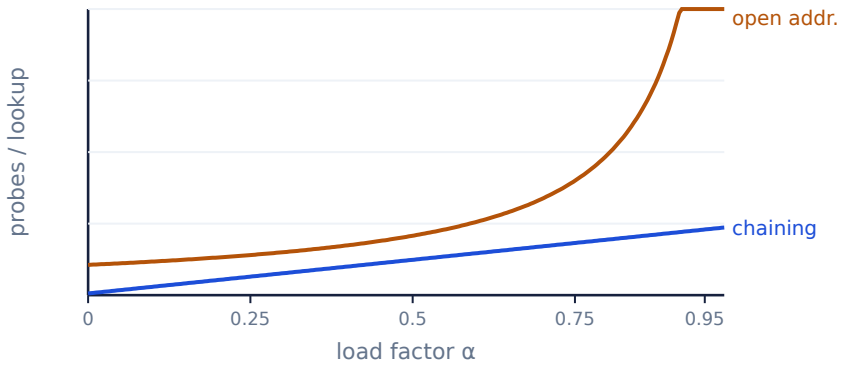

Fig 2.2 — cost against load factor. Chaining degrades linearly; open addressing has an asymptote at α = 1, which is why it must resize early.

2.2 Heaps and the O(n) build

A binary heap is a complete binary tree stored in an array: node i has children $2i{+}1$ and $2i{+}2$ and parent $(i{-}1)/2$. “Complete” means every level is full except possibly the last, which fills left to right — that is what makes the array packing possible with no gaps.

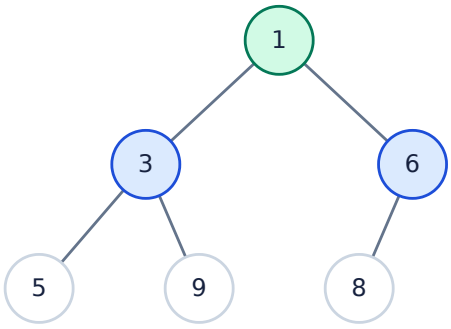

Fig 2.3 — a min-heap. Stored as `[1,3,6,5,9,8]`; the heap property is local (parent ≤ child), not a total order.

Inserting appends at the end and **sifts up** while smaller than its parent: O(log n). Extracting the minimum moves the last element to the root and **sifts down**: also O(log n). The interesting operation is building a heap from an unsorted array.

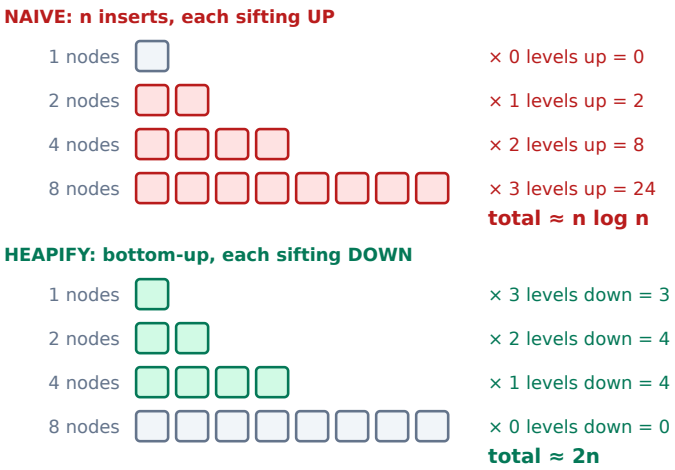

Fig 2.4 — the same 15 nodes. Sifting *up* costs most where the nodes are (the bottom); sifting *down* costs least there. Half the nodes are leaves and do no work at all.

```
// Bottom-up heapify: O(n), not O(n log n).
// Start at the last internal node — every node after it is a leaf.
for (int i = n / 2 - 1; i >= 0; --i) sift_down(a, i, n);

void sift_down(vector<int>& a, int i, int n) {
    while (true) {
        int small = i, l = 2*i + 1, r = 2*i + 2;
        if (l < n && a[l] < a[small]) small = l;
        if (r < n && a[r] < a[small]) small = r;
        if (small == i) break;
        swap(a[i], a[small]);
        i = small;
    }
}
```

THE FOLLOW-UP

“If `build_heap` is O(n), why is heapsort O(n log n)?” Because building is only the first phase. The second phase extracts the maximum n times, and each extraction sifts down O(log n) from the root — that phase genuinely is $\Theta(n \log n)$ and dominates. The O(n) build does not help the sort; it helps when you only need the *heap*.

2.3 Union-Find with two optimisations

Disjoint Set Union answers “are these two in the same group?” and “merge these two groups”. It is the backbone of Kruskal's MST, cycle detection in undirected graphs, and every connectivity problem. Naively it is O(n) per operation; two independent optimisations bring it to near-constant.

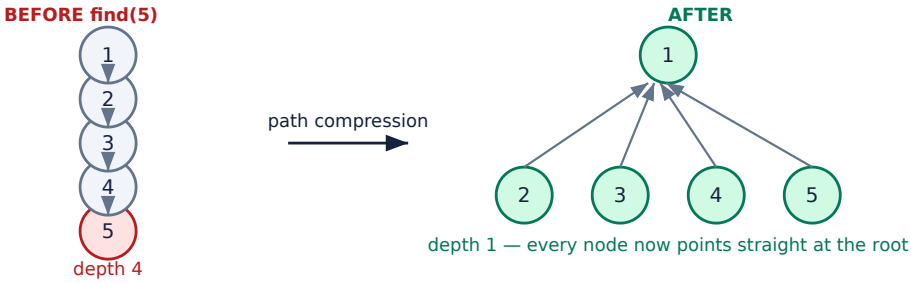

Fig 2.5 — path compression flattens the whole path on the way back up from a `find`. The cost is paid once; every later query is O(1).

```
struct DSU {
    vector<int> parent, rank_;
    DSU(int n) : parent(n), rank_(n, 0) { iota(parent.begin(), parent.end(), 0); }

    int find(int x) {                                  // path compression
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    }
    bool unite(int a, int b) {                         // union by rank
        a = find(a); b = find(b);
        if (a == b) return false;                      // already together — a cycle
        if (rank_[a] < rank_[b]) swap(a, b);           // hang the shorter under the taller
        parent[b] = a;
        if (rank_[a] == rank_[b]) ++rank_[a];
        return true;
    }
};
```

Optimisations used	Amortized per operation
neither	O(n)
union by rank / size only	O(log n)
path compression only	O(log n) amortized
both	**O(α(n))** — inverse Ackermann, < 5 for any real n

WHAT α(n) ACTUALLY MEANS

α is the inverse Ackermann function, and it grows so slowly that $\alpha(n) \le 4$ for every n below the number of atoms in the observable universe. It is constant for all practical purposes but *provably* not constant — Tarjan proved the bound is tight. Saying "effectively O(1), formally O(α(n))" is exactly right; saying "it's O(1)" invites a correction.

UNION BY RANK AND PATH COMPRESSION FIGHT

Path compression destroys the ranks it flattens — after compressing, the stored rank is an upper bound, not the real height. That is fine and intended: ranks are never recomputed. If you switch to **union by size** the sizes stay exact, which is why size is preferred when you also need "how big is this component?". Both give the same α(n) bound.

2.4 Segment trees

A segment tree answers range queries and point updates in O(log n) each, for any *associative* operation — sum, min, max, gcd, matrix product. Each node stores the answer for an interval; the root covers everything, leaves cover single elements.

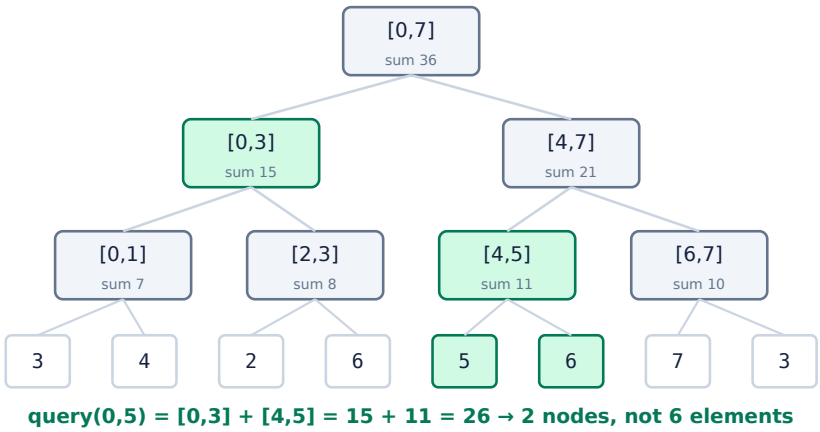

Fig 2.6 — a sum segment tree over 8 elements. Any range decomposes into at most 2 log n precomputed nodes (green).

The key insight for the interview: **any interval decomposes into O(log n) canonical nodes**. At each level the query touches at most two nodes that are partially covered; everything strictly inside is answered by one node lookup. That is where the log comes from.

Lazy propagation

Point updates are easy. *Range* updates — "add 5 to everything in [3, 9]" — would touch O(n) leaves. Lazy propagation stores the pending update at the highest node fully covered by the range and pushes it down only when a later query needs to descend through it.

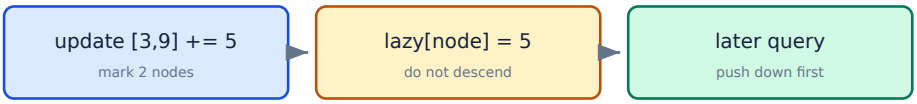

Fig 2.7 — lazy propagation defers work to the moment it is observed, keeping range updates at O(log n).

SEGMENT TREE OR FENWICK?

Reach for a **Fenwick tree** when the operation is invertible (sum, xor) and you need point update + prefix query: it is 4× less code, uses n integers rather than $4n$, and has better constants. Reach for a **segment tree** when the operation has no inverse (min, max, gcd), when you need range updates with lazy propagation, or when you need to descend the tree to find something ("first index with prefix sum ≥ k"). Saying this trade-off unprompted is a strong signal.

2.5 Fenwick trees and the low-bit trick

A Binary Indexed Tree stores, at index i, the sum of the `i & -i` elements ending at i. That expression isolates the lowest set bit, and it is the entire structure — there is no tree in memory, only an array and two loops.

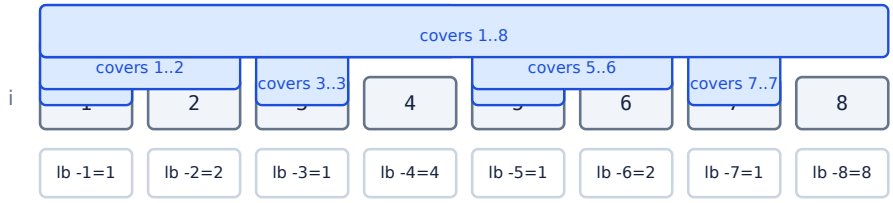

Fig 2.8 — each BIT slot covers a range whose length is its own lowest set bit. Index 8 covers all of 1–8; index 7 covers only itself.

```
struct BIT {
    vector<long long> t; int n;
    BIT(int n) : t(n + 1, 0), n(n) {}

    void add(int i, long long v) {           // 1-indexed
        for (; i <= n; i += i & -i) t[i] += v;      // climb to the covering slots
    }
    long long sum(int i) {                   // prefix sum of 1..i
        long long s = 0;
        for (; i > 0; i -= i & -i) s += t[i];       // peel off one set bit at a time
        return s;
    }
    long long range(int l, int r) { return sum(r) - sum(l - 1); }
};
```

Both loops run once per set bit in the index, so both are O(log n). The subtraction in `range` is why the operation must be invertible: there is no way to “subtract” a minimum.

2.6 Tries

A trie stores strings by shared prefix: the path from the root spells the key. Lookup is O(L) in the key length and, crucially, *independent of how many keys are stored* — which is what a hash table cannot offer for prefix queries.

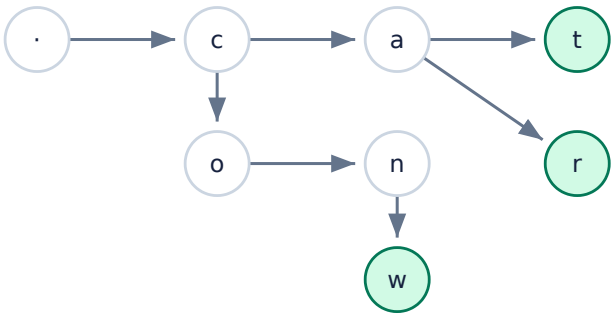

Fig 2.9 — a trie holding `cat`, `car`, `cow`. Green nodes are word ends. `ca` is stored once and shared.

Operation	Trie	Hash set	Sorted array
Exact lookup	O(L)	O(L) average	O(L log n)
Prefix search	**O(L)**	O(n·L) — scan everything	O(L log n)
All keys with prefix	**O(L + k)**	O(n·L)	O(L log n + k)
Lexicographic order	**free** — DFS	impossible	free
Memory	high — 26 pointers/node	low	lowest

THE MEMORY OBJECTION, AND THE ANSWER

A 26-child array per node wastes enormous space on sparse tries. Three standard answers: use a **hash map per node** (space proportional to real children); use a **ternary search trie** (3 pointers per node); or **compress chains** into a radix tree / Patricia trie, where a node with one child is merged into its parent. Radix trees are what routing tables and `std::filesystem`-style prefix indexes actually use.

2.7 Monotonic stacks and deques

A monotonic stack keeps its contents sorted, discarding elements that can never again be the answer. It solves the whole “next greater element” family in O(n) despite looking quadratic.

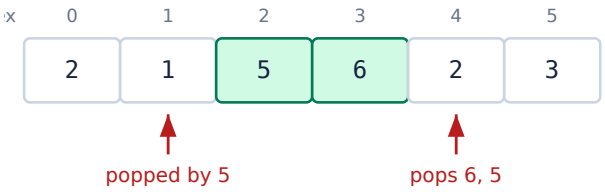

Fig 2.10 — `[2,1,5,6,2,3]`. When 5 arrives it pops 1; when 2 arrives it pops 6 and 5. Each element is pushed once and popped at most once.

```cpp
// Next greater element to the right; -1 when none exists.
vector<int> next_greater(const vector<int>& a) {
    int n = a.size();
    vector<int> res(n, -1);
    stack<int> st;                          // holds INDICES, values decreasing
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.top()] < a[i]) {
            res[st.top()] = a[i];           // a[i] is the answer for that index
            st.pop();
        }
        st.push(i);
    }
    return res;                             // O(n): each index pushed once, popped once
}
```

THE COMPLEXITY QUESTION THAT TRAPS PEOPLE

“There's a while loop inside a for loop — isn't that $O(n^2)$?” No, and the aggregate argument from §1.2 is the answer: **each index is pushed exactly once and popped at most once**, so the total number of inner iterations across the whole run is at most n. Nested loops bound the work only when the inner loop restarts; here it consumes a budget that is never replenished.

The **monotonic deque** is the sliding-window variant: keep indices in decreasing order of value, evict from the front when they leave the window, and the front is always the window maximum. That gives sliding-window maximum in O(n) rather than the O(n log n) a multiset would cost.

2.8 Interview questions

Q9 Design a data structure with O(1) insert, delete and getRandom.

A hash map alone cannot do `getRandom` (no indexable storage); a vector alone cannot do O(1) delete. **Combine them**: a `vector<int>` holding the values, and an `unordered_map<int,int>` from value to its index in the vector.

- **insert** — push onto the vector, record the index. O(1).
- **getRandom** — index the vector with `rand() % size`. O(1).
- **delete** — the trick: **swap the target with the last element**, update that element's index in the map, then `pop_back`. Deleting from the middle of a vector is O(n); deleting from the end is O(1), so move the victim to the end first.

```
bool remove(int val) {
    auto it = idx.find(val);
    if (it == idx.end()) return false;
    int i = it->second, last = data.back();
    data[i] = last; idx[last] = i;         // move last into the hole
    data.pop_back(); idx.erase(it);        // then drop the end
    return true;
}
```

Follow-up: *allow duplicates.* Store `unordered_map<int, unordered_set<int>>` mapping a value to the set of indices holding it, and be careful when the swapped element equals the removed one.

Q10 Why is std::map O(log n) but std::unordered_map O(1)? When would you still choose map?

`std::map` is a red-black tree: ordered, so every operation walks a path of height O(log n). `std::unordered_map` is a hash table: the bucket is computed arithmetically, so lookup is O(1) *average* — worst case O(n) under collisions.

Choose `map` when you need: **ordered iteration**; **range queries** (`lower_bound` / `upper_bound` — a hash map cannot do these at all); **worst-case guarantees** rather than average; **keys with no good hash**; or **iterator stability under insertion**. Also note `map` has predictable latency, while a hash map occasionally stalls for a full rehash — which matters in real-time code.

In practice, for small n (say under 100) a sorted `vector` with binary search beats both, because it is contiguous and cache-friendly.

Q11 Implement an LRU cache with O(1) get and put.

Two structures again: a **doubly linked list** in recency order (most recent at the head) and a **hash map** from key to the list node.

- **get** — look up the node in O(1), unlink it, move it to the head. O(1).
- **put** — insert at the head; if over capacity, evict the tail. O(1).

The doubly linked list is essential: unlinking a node in O(1) needs its predecessor, and a singly linked list would force an O(n) walk to find it. In C++ this is `std::list` plus `unordered_map<int, list<pair<int,int>>::iterator>`, relying on the fact that `list` iterators stay valid across splices.

Follow-up — **LFU**: keep a frequency map plus, for each frequency, its own LRU list, and track the current minimum frequency. Both operations stay O(1); the minimum can only increase by one on access, or reset to 1 on insertion.

Q12 Find the k largest elements in a stream of unknown length.

Keep a **min-heap of size k**. For each new element: if the heap has fewer than k, push it; otherwise, if the element exceeds the heap's minimum, pop and push. The heap always holds the k largest seen so far, and its root is the k-th largest.

Time O(n log k), space O(k) — and space is the point, since the stream does not fit in memory. Note the inversion that catches people out: **k largest needs a min-heap**, because the element you must be able to evict cheaply is the smallest of the ones you kept.

If the whole array is available and you only need the k-th, **quickselect** is O(n) average, and `std::nth_element` implements it. If you need all k in sorted order, the heap is usually the better answer anyway.

Q13 How does a segment tree handle range updates without touching every leaf?

Lazy propagation. When an update covers a node's interval completely, apply it to that node's aggregate and record the pending operation in a `lazy[]` array instead of recursing. Nothing below is touched.

Correctness comes from one discipline: **push down before descending**. Any traversal that needs to go below a node first applies that node's pending operation to both children and clears it. Every node is therefore consistent at the moment it is read.

The subtlety is composing operations. Range-assign and range-add do not commute, so a tree supporting both must store a composed lazy value with a defined order (assignment cancels a pending add; an add after an assignment stacks). Getting that composition wrong is the classic lazy-propagation bug.

Q14 Given a stream, report the median at any point.

Two heaps. A max-heap for the lower half, a min-heap for the upper half, kept balanced so their sizes differ by at most one.

- **insert** — push into one heap, move its extreme to the other, then rebalance. O(log n).
- **median** — if the sizes differ, the larger heap's root; otherwise the average of the two roots. O(1).

The invariant to state out loud: *every element in the max-heap is ≤ every element in the min-heap*. That is what makes the roots the two middle elements.

Follow-ups: if values are bounded integers (say ages 0–120), a counting array plus a running total gives O(1) insert and O(120) median — better constants. If you need arbitrary percentiles, an order-statistic tree or a Fenwick tree over the value range is the right structure.

Q15 Union-Find is O(α(n)). What is α, and would you say it is O(1)?

α is the **inverse Ackermann function**. Ackermann grows unimaginably fast, so its inverse grows unimaginably slowly: α(n) ≤ 4 for every n up to roughly $2\uparrow\uparrow65536$, far beyond any physical input.

So: *"effectively constant, formally O(α(n))"*. Do not just say O(1) — Tarjan proved the bound is **tight**, so there genuinely is no constant-time bound for the pointer-machine model. Claiming O(1) invites a correction; naming α and immediately adding "which is at most 4 in practice" shows you know both the theory and that it does not matter operationally.

Note the bound needs *both* path compression and union by rank/size. Either alone gives only O(log n).

Q16 Why does a B-tree beat a red-black tree for a database index?

Because the cost model is completely different. In RAM, a comparison and a pointer dereference cost about the same, so minimising comparisons (a binary tree) is right. On disk or SSD, a **single random read costs 10^5–10^6× a comparison**, so the goal is minimising *node visits*, not comparisons.

A B-tree packs hundreds of keys into one node sized to the disk page (typically 4–16 KB). With a branching factor of ~500, a billion records fit in **four levels** — four disk reads. A red-black tree over the same data is ~30 levels deep and would cost 30 random reads.

The same argument explains B+ trees specifically: keeping all values in the leaves and linking the leaves makes range scans sequential, which is the access pattern databases care about most.

Q17 Design a data structure supporting insert, delete, and getMin in O(1).

A heap gives O(log n) delete, so it is not enough. Use **two stacks**, or one stack of pairs: alongside each value store the minimum of the stack *at that point*. Push compares the new value against the current minimum and stores the smaller; `getMin` reads the top's second field. All three operations are O(1), at the cost of O(n) extra space.

```
struct MinStack {
    stack<pair<int,int>> s;                          // {value, min so far}
    void push(int v) { s.push({v, s.empty() ? v : min(v, s.top().second)}); }
    void pop()       { s.pop(); }
    int  top()       { return s.top().first; }
    int  getMin()    { return s.top().second; }
};
```

This works because a stack only removes the most recent element, so the history of minima is exactly a stack too. The constraint that makes it possible is **LIFO**: implement the same thing for a queue and you need two stacks with amortized analysis, and for arbitrary deletion you need a heap or a balanced tree.

Q18 You need range-sum queries on an immutable array. What do you use?

A prefix-sum array — not a segment tree. Build `P[i] = a[0] + … + a[i-1]` once in O(n), then any range sum is `P[r+1] - P[l]` in **O(1)**. The segment tree's O(log n) is only worth paying for when updates exist.

Recognising that immutability removes the need for the fancy structure is the point of the question. The 2-D version generalises: a 2-D prefix sum answers submatrix queries in O(1) with inclusion–exclusion, `S[r2][c2] - S[r1-1][c2] - S[r2][c1-1] + S[r1-1][c1-1]`.

If updates are added later: point update + prefix query → Fenwick tree; range update + point query → Fenwick over a difference array; both → two Fenwick trees, or a lazy segment tree.

Q19 Why can't a hash map answer 'give me all keys between 100 and 200'?

Because hashing **deliberately destroys order**. A good hash function distributes keys uniformly, which means keys 100 and 101 land in unrelated buckets by design — that is precisely the property that prevents clustering. There is no way to enumerate a range without scanning every bucket, which is O(n).

The structures that *can*: a balanced BST (`std::map`) via `lower_bound`, walking in order — O(log n + k); a sorted array via binary search — same bound, better constants, no updates; a B+ tree — the same idea tuned for disk; a skip list — the same bound with simpler concurrent updates, which is why Redis uses one for sorted sets.

An *order-preserving* hash exists but reintroduces exactly the clustering that hashing was meant to avoid, so it is rarely the answer.

Q20 What is the difference between a heap and a BST, and when is a heap the wrong choice?

A heap enforces a **local** property (parent ≤ both children) and gives you O(1) access to one extreme. A BST enforces a **global** ordering, so it can answer search, predecessor, successor and range queries — all of which a heap does in O(n).

A heap is the wrong choice whenever you need more than the extreme: searching for an arbitrary value, deleting an arbitrary value (O(n) to find it first), iterating in sorted order, or answering “what is the second largest?” repeatedly.

What a heap gives in exchange: O(n) construction against O(n log n), an array layout with no pointers, and O(1) peek. For scheduling, Dijkstra, top-k and merging sorted streams, that trade is exactly right.

PART 3

Graph Algorithms

Recognising the graph inside the word problem, then choosing the cheapest algorithm that is still correct.

Graph questions dominate senior interviews because almost every real system is a graph: dependencies, routing, social connections, build order, scheduling. The algorithms are standard; what is tested is whether you recognise the graph hiding inside a word problem and pick the cheapest algorithm that is still correct.

3.1 Representation trade-offs

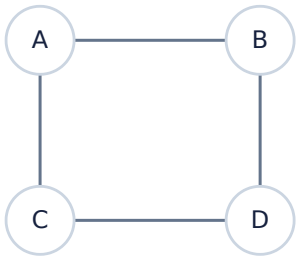

Fig 3.1 — four vertices, four edges.

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

Adjacency matrix — O(V²) space, O(1) edge test.

	Adjacency list	Adjacency matrix	Edge list
Space	O(V + E)	O(V²)	O(E)
Is (u,v) an edge?	O(deg u)	**O(1)**	O(E)
Iterate neighbours of u	**O(deg u)**	O(V)	O(E)
Add an edge	O(1)	O(1)	O(1)
Best for	sparse — almost everything	dense, or Floyd-Warshall	Kruskal, sorting edges

SPARSE IS THE DEFAULT

Real graphs are overwhelmingly sparse: E is closer to V than to V². Road networks have average degree ~3; social graphs have heavy tails but still E ≪ V². Default to an adjacency list and say so. A matrix over 10^5 vertices is 10^{10} entries — it does not fit, and mentioning that shows you did the arithmetic.

3.2 BFS, DFS and edge classification

Both visit every vertex and edge once, so both are O(V + E). The difference is the order, and the order is what makes each useful.

	BFS	DFS
Frontier	queue	stack / recursion
Visits	level by level	as deep as possible first
Shortest path	**yes, on unweighted graphs**	no
Space	O(V) — a whole level	O(V) — path depth
Natural for	shortest hops, level order, bipartite check	cycles, topological order, SCC, backtracking

Running DFS on a directed graph classifies every edge, and that classification is the foundation of cycle detection, topological sort and SCC.

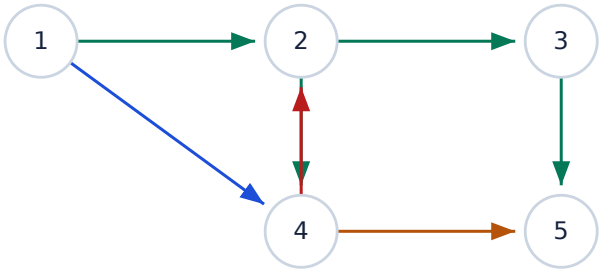

Fig 3.2 — DFS from 1. **Green** tree edges · **red** back edge (proves a cycle) · **blue** forward edge · **amber** cross edge.

THE CYCLE-DETECTION RULE

A directed graph has a cycle if and only if DFS finds a back edge — an edge to a vertex currently on the recursion stack (colour *grey* in the white/grey/black scheme). Note the precision required: an edge to an already-*visited* vertex is not enough, because forward and cross edges do that too and are harmless. You need "still on the stack", which needs three states, not a boolean.

In an **undirected** graph the rule is different: any edge to a visited vertex that is not the parent you came from closes a cycle.

```
// Directed cycle detection — three colours, not a visited flag.
enum { WHITE, GREY, BLACK };
bool dfs(int u, vector<int>& colour, vector<vector<int>>& adj) {
    colour[u] = GREY;                                  // on the recursion stack
    for (int v : adj[u]) {
        if (colour[v] == GREY) return true;            // back edge — cycle
        if (colour[v] == WHITE && dfs(v, colour, adj)) return true;
    }
    colour[u] = BLACK;                                 // finished
    return false;
}
```

3.3 Dijkstra — and exactly why negatives break it

Dijkstra grows a set of vertices whose shortest distance is final. At each step it takes the unfinalised vertex with the smallest tentative distance, declares it final, and relaxes its outgoing edges. With a binary heap that is **O((V + E) log V)**.

The correctness argument is one sentence, and it is the sentence the interviewer wants: *when a vertex is popped with distance d, no shorter path to it can exist, because any other path must leave the finalised set through a frontier vertex whose distance is already $\geq d$, and edges only add non-negative weight.* The moment weights can be negative, that argument collapses.

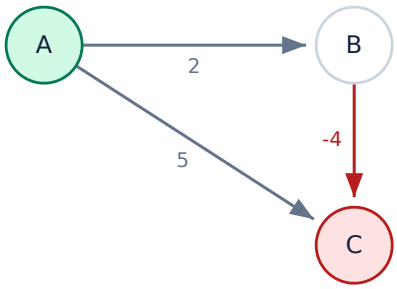

Fig 3.3 — the counterexample. Dijkstra finalises C at 5 (direct edge) before exploring B, but A→B→C costs $2 - 4 = \mathbf{-2}$. The finalised distance is simply wrong.

Algorithm	Time	Negative weights	Negative cycles	Use when
BFS	O(V + E)	n/a	n/a	unweighted — do not reach for Dijkstra
Dijkstra (heap)	O((V+E) log V)	**no**	no	non-negative, single source
0-1 BFS (deque)	O(V + E)	no	no	weights are only 0 or 1
Bellman-Ford	O(V·E)	**yes**	**detects**	negatives possible
SPFA	O(V·E) worst	yes	detects	usually fast, bad worst case
Floyd-Warshall	O(V³)	yes	detects	all pairs, dense, V ≤ ~500
Johnson	O(V·E + V·E log V)	yes	detects	all pairs, sparse

```cpp
// Dijkstra with a lazy heap: push duplicates, skip stale pops.
vector<long long> dijkstra(int src, const vector<vector<pair<int,int>>>& adj) {
    const long long INF = LLONG_MAX / 4;
    vector<long long> dist(adj.size(), INF);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;                  // stale entry — the key trick
        for (auto [v, w] : adj[u])
            if (d + w < dist[v]) {
                dist[v] = d + w;
                pq.push({dist[v], v});              // no decrease-key needed
            }
    }
    return dist;
}
```

WHY NOT decrease-key?

The textbook Dijkstra uses `decrease_key`, which `std::priority_queue` does not support. The **lazy** variant above pushes a duplicate instead and discards stale entries on pop (`if (d > dist[u]) continue;`). The heap can hold O(E) entries rather than O(V), so the bound is O(E log E) — but log E ≤ 2 log V, so it is the same O(E log V) asymptotically, with far simpler code. Fibonacci heaps give a theoretical O(E + V log V) that is almost never faster in practice.

3.4 Bellman-Ford and Floyd-Warshall

Bellman-Ford relaxes every edge V−1 times. The bound is exact: a shortest path in a graph with no negative cycle has at most V−1 edges, and after k rounds every shortest path using ≤ k edges is correct. If a V-th round still improves something, a negative cycle is reachable — that is the detection mechanism, not an add-on.

Fig 3.4 — Bellman-Ford by rounds. The V-th round is the negative-cycle test.

Floyd-Warshall is three nested loops and the cleanest DP in the book: `d[i][j] = min(d[i][j], d[i][k] + d[k][j])`, where k ranges over "intermediate vertices allowed so far". **The k loop must be outermost** — it is the DP dimension being built up, and swapping the loops silently produces wrong answers on some graphs.

```
// Floyd-Warshall. k OUTERMOST — this is the DP layer, not just a loop.
for (int k = 0; k < n; ++k)
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

for (int i = 0; i < n; ++i)
    if (d[i][i] < 0) { /* i lies on a negative cycle */ }
```

3.5 Topological order and cycle detection

A topological order lists vertices so every edge points forward. It exists **if and only if** the graph is a DAG, which is why the same algorithm answers "is there a cycle?" and "in what order can I build this?".

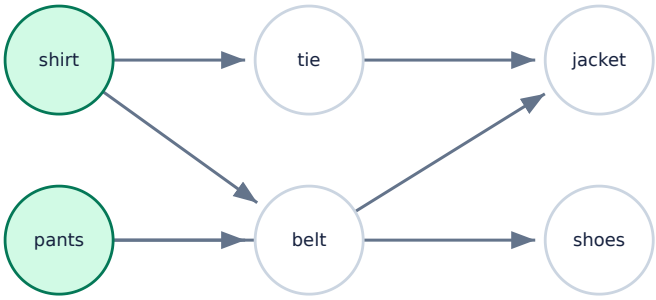

Fig 3.5 — a dependency DAG. Green vertices have in-degree 0, so they are the only valid starting points. Several orders are valid; topological order is not unique.

```cpp
// Kahn's algorithm — BFS on in-degrees. Detects cycles for free.
vector<int> topo_sort(int n, const vector<vector<int>>& adj) {
    vector<int> indeg(n, 0), order;
    for (int u = 0; u < n; ++u) for (int v : adj[u]) ++indeg[v];

    queue<int> q;
    for (int u = 0; u < n; ++u) if (indeg[u] == 0) q.push(u);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    // Fewer than n vertices emitted ⇒ the remainder is trapped in a cycle.
    return order.size() == (size_t)n ? order : vector<int>{};
}
```

THE INTERVIEW FRAMING

Course Schedule, build systems, task scheduling with dependencies, package managers, spreadsheet recalculation, and deadlock detection are all this question. If you use a **priority queue** instead of a plain queue you get the lexicographically smallest valid order — a very common follow-up. The DFS variant produces the reverse postorder instead, and is the basis of Tarjan's SCC below.

3.6 Strongly connected components

In a directed graph, u and v are strongly connected when each can reach the other. SCCs partition the vertices, and contracting each SCC to a single node always yields a DAG — the **condensation**. That fact is the real payoff: it turns any directed graph into a DAG you can run DP on.

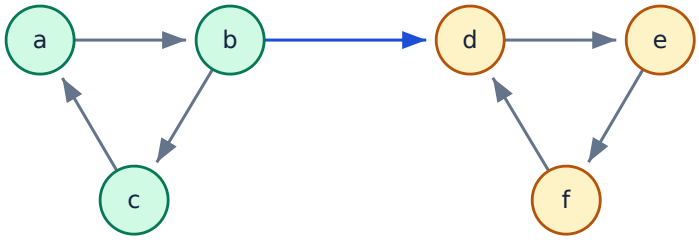

Fig 3.6 — two SCCs. The blue edge is the only one between them, so the condensation is a two-node DAG — and no edge ever returns.

	Kosaraju	Tarjan
Passes over the graph	two DFS passes	**one** DFS pass
Needs the reversed graph	**yes**	no
Extra state	finish-order stack	`disc[]`, `low[]`, on-stack flag
Complexity	O(V + E)	O(V + E)
Easier to derive live	**yes**	no — the low-link rule is subtle

WHICH TO OFFER

Say Kosaraju if asked to *derive* one: "DFS recording finish times, reverse every edge, then DFS again in decreasing finish order — each tree is an SCC." The intuition is clean (the reversed graph keeps SCCs intact but flips the cross-component edges, so a second DFS cannot escape). Say Tarjan if performance matters — one pass, no reversed copy — and be ready to explain `low[u] = min(low[u], disc[v])` for on-stack v, `min(low[u], low[v])` for tree edges, with `low[u] == disc[u]` identifying an SCC root.

3.7 Bridges and articulation points

A **bridge** is an edge whose removal disconnects the graph; an **articulation point** is such a vertex. Both fall out of the same DFS low-link machinery, and both are network-reliability questions in disguise: single points of failure.

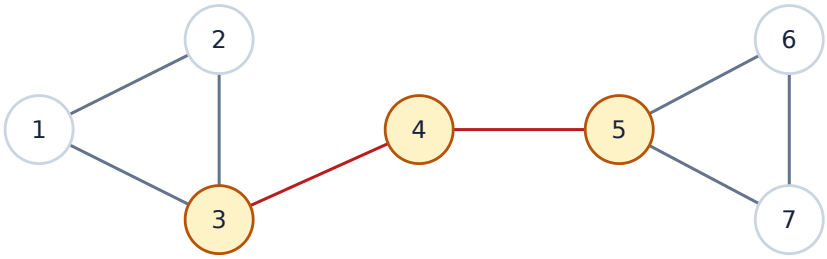

Fig 3.7 — red edges are bridges; amber vertices are articulation points. Removing edge 3–4 splits the graph in two.

The rule: keep `disc[u]` (discovery time) and `low[u]` (the earliest discovery time reachable from u's subtree using at most one back edge). For a tree edge $u \to v$, the edge is a **bridge** when `low[v]`

`> disc[u]` — the subtree cannot reach above u at all. u is an **articulation point** when `low[v] ≥ disc[u]` for some child, with the root special-cased: it is an articulation point exactly when it has more than one DFS child.

3.8 Minimum spanning trees

	Kruskal	Prim
Approach	sort edges, add if no cycle	grow one tree, add cheapest frontier edge
Needs	Union-Find	priority queue
Time	O(E log E)	O(E log V) with a heap; O(V²) with an array
Best for	sparse graphs, edge list given	dense graphs
Handles disconnection	**yes — yields a forest**	no — one component only

THE CUT PROPERTY — WHY BOTH ARE CORRECT

For any partition of the vertices into two sets, **the minimum-weight edge crossing that cut belongs to some MST**. Kruskal applies it globally (the cheapest remaining edge crosses the cut between the components it joins); Prim applies it to the cut between the built tree and everything else. One property, two greedy strategies — being able to say this is what distinguishes understanding from memorisation.

Corollary worth knowing: if all edge weights are **distinct**, the MST is **unique**. Duplicated weights are what allow multiple MSTs.

3.9 Max-flow and min-cut

The max-flow min-cut theorem states that the maximum flow from source to sink equals the minimum total capacity of any set of edges whose removal disconnects them. It is the most reliably useful “advanced” result in interviews, because so many problems reduce to it: bipartite matching, project selection, image segmentation, and any “maximum number of disjoint paths” question.

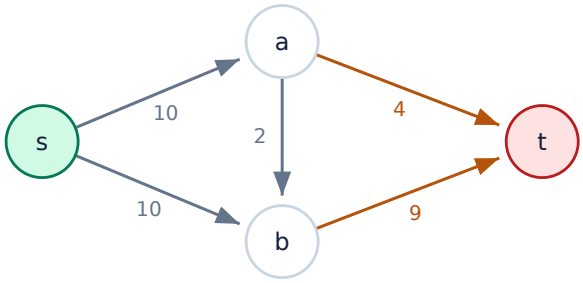

Fig 3.8 — max flow = 13. The min cut is the amber pair {a→t, b→t} with capacity 4 + 9 = 13 — equal, as the theorem requires.

Algorithm	Time	Notes
Ford-Fulkerson	O(E · maxflow)	may not terminate on irrational capacities
Edmonds-Karp	$O(V \cdot E^2)$	BFS for the augmenting path — guarantees termination
Dinic	$O(V^2E)$, $O(E\sqrt{V})$ unit caps	the practical default; matching in $O(E\sqrt{V})$

THE REDUCTION TO SPOT

Maximum bipartite matching is max-flow: add a source joined to every left vertex and a sink joined from every right vertex, give every edge capacity 1, and the max flow is the matching size. Kőnig's theorem then gives you *minimum vertex cover = maximum matching* in bipartite graphs, and *maximum independent set = V − maximum matching*. A surprising number of "assign tasks to workers" and "select non-conflicting items" problems are exactly this.

3.10 Interview questions

Q21 Find the shortest path in a grid with obstacles. Which algorithm?

BFS, not Dijkstra. Every move costs 1, so the graph is unweighted and BFS gives the shortest path in O(V + E) = O(rows × cols). Using Dijkstra here is not wrong but adds a needless log factor, and interviewers notice.

The follow-ups change the answer, and knowing which is which is the real test:

- **Different terrain costs** → Dijkstra.
- **Costs are only 0 or 1** (e.g. breaking a wall is free/costly) → **0-1 BFS** with a deque: push 0-cost moves to the front, 1-cost to the back. O(V + E), no heap.
- **"You may remove up to k obstacles"** → BFS over an expanded state space `(row, col, obstacles_removed)`. The insight is that the *state* carries the budget; the algorithm does not change.
- **Single target, geometric grid** → A* with the Manhattan distance heuristic prunes hard while staying optimal, since that heuristic is admissible.

Q22 Detect a cycle in a directed graph, then in an undirected one. Are they the same?

No, and using the directed method on an undirected graph is a common bug.

Directed: DFS with three colours. A cycle exists exactly when you find an edge to a *grey* vertex — one still on the recursion stack. A boolean visited flag is insufficient: cross edges and forward edges also point at visited vertices and are not cycles.

Undirected: a cycle exists when you find an edge to a visited vertex that is *not the parent* you arrived from. Every edge is bidirectional, so without the parent check every single edge looks like a 2-cycle. Careful with parallel edges: two distinct edges between the same pair *are* a cycle, so track the edge id rather than the parent vertex when multi-edges are allowed.

Alternatives: Kahn's algorithm detects directed cycles (fewer than n vertices emitted); Union-Find detects undirected cycles (an edge whose endpoints already share a root) in $O(E\ \alpha(V))$.

Q23 Why can't Dijkstra handle negative edges? Would adding a constant to every weight fix it?

Dijkstra's correctness rests on a monotonicity assumption: once a vertex is popped with distance d, no later discovery can beat it, because every remaining path leaves through a frontier vertex already at distance ≥ d and edges only add. A negative edge violates that — a longer-looking detour can end up cheaper. Fig 3.3 is the two-line counterexample.

Adding a constant does not fix it, and this is the real question. Adding c to every edge adds $c \times$ *(number of edges on the path)*, not a constant per path. It therefore penalises paths with more edges and can change which path is shortest. In Fig 3.3, adding 4: A→C becomes 9, A→B→C becomes 6 + 0 = 6 — still the shorter, by luck; adjust the numbers slightly and the ranking flips.

The correct reweighting is **Johnson's algorithm**: run Bellman-Ford from a virtual source to get a potential $h(v)$, then set `w'(u,v) = w(u,v) + h(u) − h(v)`. This is non-negative and, because the h terms telescope along any path, it preserves the shortest-path ordering exactly. Then Dijkstra is safe.

Q24 Given course prerequisites, determine whether all courses can be finished, and in what order.

Topological sort. Build a graph with an edge prerequisite → course, then run Kahn's algorithm: repeatedly take a vertex with in-degree 0, emit it, and decrement its neighbours.

If fewer than n vertices come out, the remainder sits in a cycle and the courses cannot all be finished — the same run answers both questions, which is worth pointing out. O(V + E).

Follow-ups: **lexicographically smallest order** → a priority queue instead of a queue, O((V+E) log V). **Minimum number of semesters** with unlimited parallel courses → the length of the longest path in the DAG, computed by processing in topological order — equivalently, the number of BFS levels in Kahn's algorithm. **Limited courses per semester** → NP-hard in general; say so rather than inventing a greedy that fails.

Q25 Clone a directed graph with cycles.

DFS or BFS with a **hash map from original node to its copy**. The map is doing two jobs at once, and naming both is the answer: it is the visited set that terminates the traversal on cycles, and it is the lookup that lets a second reference to an already-cloned node point at the same copy rather than duplicating it.

```
Node* clone(Node* u, unordered_map<Node*, Node*>& seen) {
    if (!u) return nullptr;
    auto it = seen.find(u);
    if (it != seen.end()) return it->second;      // visited AND already-built copy
    Node* c = new Node(u->val);
    seen[u] = c;                                  // register BEFORE recursing — cycles
    for (Node* v : u->neighbors) c->neighbors.push_back(clone(v, seen));
    return c;
}
```

The critical line is `seen[u] = c;` *before* the recursion. Registering after would recurse forever on a cycle. O(V + E) time and space.

Q26 Find the number of islands in a 2-D grid, then handle a stream of additions.

Static: scan every cell; on an unvisited '1', run DFS or BFS to sink the whole island, and count how many times you start. O(rows × cols). Mention that DFS may overflow the stack on a 10^5-cell grid of all land — BFS or an explicit stack is safer.

Streaming ("Number of Islands II"): re-running the scan per addition is O(k · mn). Use **Union-Find** instead. Keep a running component count; each addition increments it, then for each of the four neighbours that is already land, attempt a union — every *successful* union decrements the count. O(k · α(mn)), essentially linear.

This is the canonical example of a problem where the offline and online versions want completely different structures, which is exactly why it gets asked.

Q27 What is the difference between Prim's and Kruskal's, and when does each win?

Both are greedy and both rest on the **cut property**. Kruskal sorts all edges and adds any that does not create a cycle, using Union-Find — O(E log E), dominated by the sort. Prim grows a single tree, repeatedly taking the cheapest edge leaving it, using a priority queue — O(E log V).

Kruskal wins on sparse graphs, when the input is already an edge list, and when the edges arrive pre-sorted (then it is O(E α(V)), effectively linear). It also handles **disconnected** graphs naturally, producing a minimum spanning *forest*.

Prim wins on dense graphs — with an adjacency matrix and a simple array instead of a heap it is $O(V^2)$, which beats O(E log E) = $O(V^2 \log V)$ when $E \approx V^2$. It also only ever touches one growing component, which suits streaming the graph in.

Q28 How would you find whether a graph is bipartite?

Two-colour it with BFS or DFS. Colour the start vertex 0, every neighbour 1, their neighbours 0, and so on. If you ever meet an edge whose endpoints already share a colour, the graph is not bipartite. O(V + E).

The theory worth stating: *a graph is bipartite if and only if it contains no cycle of odd length*. The colouring conflict you detect **is** an odd cycle — the two paths from the last common ancestor have the same parity, so together with the offending edge they close a cycle of odd length.

Run it over **every component**: a disconnected graph is bipartite only if all its components are. Applications: assigning conflicting tasks to two machines, detecting two-team splits, and unlocking the matching theory in §3.9 — Hopcroft-Karp gives maximum bipartite matching in O(E√V).

Q29 You have 10^6 vertices and 10^7 edges. Which representations are viable?

Adjacency list only. Do the arithmetic out loud — that is the point of the question.

- **Matrix:** $10^5 \times 10^5 = 10^{12}$ entries. Even at one bit each that is 125 GB. Impossible.
- **Adjacency list:** 10^6 vectors plus 2×10^7 entries (undirected). At 4 bytes per `int`, roughly 80 MB of edge data plus vector overhead. Fits comfortably.
- **CSR / flat arrays** (one `offset[]` array of size V+1 and one `target[]` array of size 2E) removes the per-vector overhead entirely and is contiguous, so traversal is dramatically more cache-friendly. This is what serious graph libraries use, and mentioning it is a differentiator.

Also flag the practical hazard at this scale: **recursive DFS will overflow the stack**. Use BFS or an explicit stack.

Q30 Find the shortest path that visits all nodes (any start, any end).

This is the **Travelling Salesman** family — NP-hard. Say that first; the interviewer is checking whether you recognise intractability rather than confidently producing a wrong polynomial algorithm.

Then give the exponential DP that is actually expected: **Held-Karp**, bitmask DP over subsets. `dp[mask][i]` = shortest walk covering the vertex set *mask* and ending at *i*. Transition `dp[mask | 1<<j][j] = min(…, dp[mask][i] + w(i,j))`. **$O(2^n \cdot n^2)$** time and $O(2^n \cdot n)$ space — feasible to about n = 20, which is exactly why `n ≤ 20` in a problem statement is a bitmask hint.

Beyond that: Christofides gives a 1.5-approximation for metric TSP; held-Karp lower bounds plus branch-and-bound handle a few hundred cities exactly; and in practice, LKH-style local search solves instances with millions of nodes to within a fraction of a percent.

Q31 Explain why BFS finds shortest paths but DFS does not.

BFS explores in **non-decreasing order of distance**. It maintains the invariant that the queue holds vertices at distance d followed by vertices at d+1 and nothing else, so the first time a vertex is dequeued, it is at its minimum possible distance. Formally: BFS dequeues vertices in non-decreasing distance order, and each vertex is enqueued exactly once, when it is first discovered — via a shortest path by induction.

DFS commits to depth. It may reach a vertex by a long path first and mark it visited, never revisiting it via the short one. Nothing in DFS orders exploration by distance.

The caveat: this only holds for **unweighted** graphs (or uniform weights), because “fewest edges” and “least weight” coincide there. With weights, the generalisation of “always expand the closest frontier vertex” is precisely Dijkstra — BFS with a priority queue instead of a FIFO.

Q32 Given a list of accounts with emails, merge those belonging to the same person.

A graph connectivity problem in disguise, and spotting that is the whole question. Treat each **email as a vertex**; within one account, union every email together. Any two accounts sharing an email end up in the same component, transitively — which is exactly the merging rule.

Union-Find is the natural fit: map each email to an integer id, union all emails within each account, then group by root. Sort each group and attach the owner's name. O(N α(N) + N log N) where the log is the final per-group sort.

DFS over the same graph works identically; Union-Find is preferred because the merging is incremental and no explicit adjacency structure is needed. The general lesson — and the reason this shows up so often — is that *“merge things that share an attribute, transitively”* is always connected components.

PART 4

Dynamic Programming

Deriving the recurrence rather than recalling it — and knowing which constraint is pointing at which technique.

Most candidates can implement a DP they have seen. Far fewer can *derive* one under pressure, and that is what the question is for. The skill is not the recurrence — it is choosing the state.

4.1 Designing the state

Dynamic programming applies when a problem has **optimal substructure** (the optimum is built from optima of subproblems) and **overlapping subproblems** (the same subproblem recurs). Without overlap you have divide and conquer; without optimal substructure you have neither.

A reliable derivation procedure, worth saying aloud in an interview:

Fig 4.1 — never start at step 4. Deriving the brute force first is what makes the state obvious, and it gives you something correct to test against.

THE STATE IS WHAT YOU NEED TO REMEMBER

Ask: *if someone paused me mid-computation, what is the minimum I would need written down to continue?* That set is the state. For knapsack it is (item index, remaining capacity). For edit distance it is (position in A, position in B). For stock trading with a cooldown it is (day, holding?, in cooldown?).

If the state is too small the recurrence will not close; if it is too big you will TLE. Growing it is the standard fix when a greedy fails: *"I need to remember one more thing."*

Symptom	State fix
Greedy fails on a counterexample	add the quantity the counterexample exploited
"at most k operations"	add a dimension for operations used
"cannot pick adjacent"	add a boolean: did I take the previous?
"n ≤ 20" over subsets	the state is a bitmask
Choices depend on a running total	the total is part of the state
Answer depends on where a range splits	state is the range (i, j) — interval DP

4.2 The knapsack family

Items with weights and values; a capacity W; maximise value. `dp[i][w]` = best value using the first i items within capacity w. Each item is either skipped or taken:

`dp[i][w] = max(dp[i-1][w], dp[i-1][w - wt[i]] + val[i])`

Fig 4.2 — items A(wt 2, val 3), B(wt 3, val 4), C(wt 4, val 5), W = 5. The answer 7 (green) comes from the two blue cells: take B (4) plus the best of capacity 2 without it (3).

```
// 2-D: O(nW) time, O(nW) space.
for (int i = 1; i <= n; ++i)
    for (int w = 0; w <= W; ++w) {
        dp[i][w] = dp[i-1][w];                                  // skip item i
        if (wt[i] <= w)
            dp[i][w] = max(dp[i][w], dp[i-1][w - wt[i]] + val[i]);   // take it
    }

// 1-D: O(W) space. The loop direction is the whole trick.
for (int i = 1; i <= n; ++i)
    for (int w = W; w >= wt[i]; --w)                            // BACKWARD for 0/1
        dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
```

THE LOOP DIRECTION IS THE 0/1 CONSTRAINT

In the 1-D version, iterating **backward** means `dp[w - wt[i]]` still holds the value from row $i-1$, so item i is used at most once. Iterating **forward** reads a cell already updated in this row, so item i can be reused — which is exactly the **unbounded** knapsack. The same three lines solve two different problems, and the only difference is a loop direction. Interviewers ask precisely because it looks like a typo.

Variant	Constraint	Loop	Complexity
0/1 knapsack	each item once	capacity **descending**	O(nW)
Unbounded	unlimited copies	capacity **ascending**	O(nW)
Bounded	at most k copies	binary-split into powers of 2	O(nW log k)
Subset sum	values = weights, hit exactly	descending	O(nW)
Partition equal subset	subset sum to total/2	descending	O(n·sum)
Coin change (min coins)	unbounded, minimise count	ascending	O(n·amount)
Coin change (count ways)	order does not matter	**coins outer**	O(n·amount)

COUNTING COMBINATIONS vs PERMUTATIONS

For “how many ways to make amount X”, the loop *nesting* decides which you count. **Coins outer, amount inner** counts each combination once ({1,2} and {2,1} are the same). **Amount outer, coins inner** counts ordered sequences, so it counts permutations. Two identical-looking loops, two different answers — read the problem statement for whether order matters.

4.3 Longest increasing subsequence in O(n log n)

The O(n²) DP is immediate: `dp[i]` = LIS ending at i, taking the best `dp[j] + 1` over all $j < i$ with `a[j] < a[i]`. The O(n log n) version is the one worth knowing, and it is worth understanding rather than memorising.

Keep an array `tails` where `tails[k]` is the **smallest possible tail** of an increasing subsequence of length k+1. That array is always sorted, so each new element can be placed by binary search: it either extends the array or improves one entry.

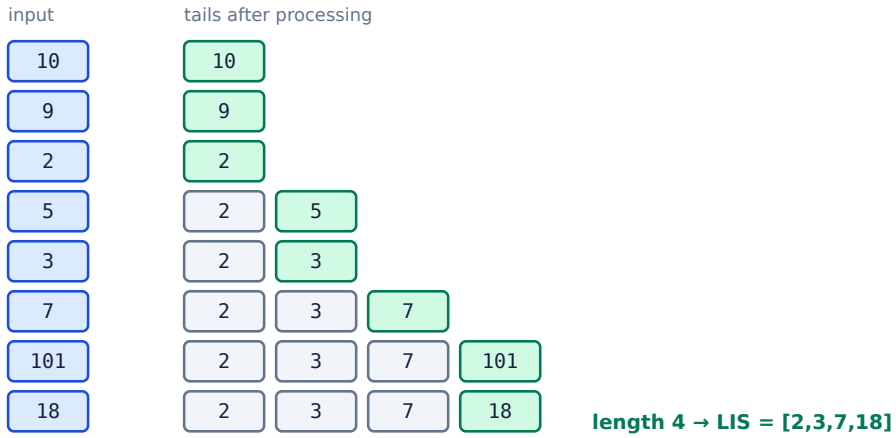

Fig 4.3 — `[10,9,2,5,3,7,101,18]`. Green is the cell written at each step. `tails` is *not* the LIS — only its length is meaningful.

```cpp
int lis(const vector<int>& a) {
    vector<int> tails;                          // tails[k] = smallest tail of an LIS of length k+1
    for (int x : a) {
        auto it = lower_bound(tails.begin(), tails.end(), x);   // strictly increasing
        if (it == tails.end()) tails.push_back(x);              // extends the longest
        else *it = x;                                           // improves a tail
    }
    return tails.size();                        // O(n log n)
}
```

THE TRAP QUESTION

"Is `tails` the actual LIS?" **No.** In Fig 4.3 the final array happens to be a valid LIS, but that is a coincidence — on `[3,4,5,1]` it ends as `[1,4,5]`, which is not increasing in the original order and is not a subsequence of anything useful. Only the *length* is guaranteed. To reconstruct the sequence, record for each element the index it was placed at plus a parent pointer, then walk back.

Use `upper_bound` instead of `lower_bound` to get the longest *non-decreasing* subsequence — another one-character difference with a different meaning.

4.4 DP on trees

Trees have no cycles, so a post-order DFS visits every subproblem exactly once — the memoisation is structural rather than explicit. The state is usually (node, some small flag), and the recurrence combines children.

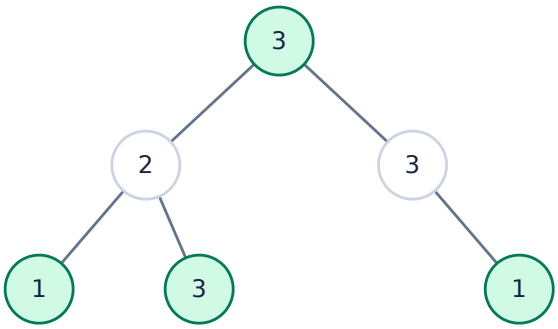

Fig 4.4 — House Robber III. Green nodes are robbed: 3 + 1 + 3 + 1 = 8 beats taking the second level (2 + 3 = 5). State = (node, was the parent taken?).

```
// Returns {best if this node is NOT taken, best if it IS taken}.
pair<int,int> rob(TreeNode* n) {
    if (!n) return {0, 0};
    auto [ln, lt] = rob(n->left);
    auto [rn, rt] = rob(n->right);
    int take  = n->val + ln + rn;              // children must then be skipped
    int skip  = max(ln, lt) + max(rn, rt);     // children free to choose
    return {skip, take};
}
```

Returning a pair instead of using a global map is the idiomatic trick: the "state" dimension becomes the tuple you return. The same pattern solves tree diameter (return depth, update a global best), maximum path sum (return the best downward path, update a global with the best through-path), and independent set on trees.

THE REROOTING TECHNIQUE

When the question asks for an answer *for every node as root* — "sum of distances from each vertex to all others" — the naive approach is O(n²). **Rerooting** does it in O(n): one post-order pass computes each subtree's contribution, then a pre-order pass transforms a parent's answer into each child's by moving the root across one edge. Recognising a rerooting problem is a strong senior signal.

4.5 Bitmask DP

When $n \le 20$–22, a subset of the n items fits in an integer and the state can be "which subset have I used?". The constraint is the hint: $2^{20} \approx 10^6$ states is comfortable, $2^{25} \approx 3\times10^7$ is the practical ceiling.

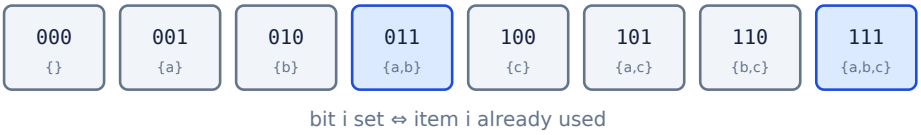

Fig 4.5 — three items, eight subsets. Iterating `mask` from 0 upward guarantees every submask is computed before the mask that needs it.

```cpp
// Held-Karp / TSP: dp[mask][i] = cheapest route covering `mask`, ending at i.
vector<vector<int>> dp(1 << n, vector<int>(n, INF));
dp[1][0] = 0;                                          // start at city 0
for (int mask = 1; mask < (1 << n); ++mask)
    for (int i = 0; i < n; ++i) {
        if (!(mask & (1 << i)) || dp[mask][i] == INF) continue;
        for (int j = 0; j < n; ++j) {
            if (mask & (1 << j)) continue;             // j already visited
            int nxt = mask | (1 << j);
            dp[nxt][j] = min(dp[nxt][j], dp[mask][i] + cost[i][j]);
        }
    }                                                  // O(2^n · n^2)
```

Bit trick	Meaning
`mask & (1 << i)`	is item *i* in the set?
`mask | (1 << i)`	add item *i*
`mask & ~(1 << i)`	remove item *i*
`mask & (mask - 1)`	clear the lowest set bit
`mask & -mask`	isolate the lowest set bit
`__builtin_popcount(mask)`	how many items are in the set
`for (s = m; s; s = (s-1) & m)`	iterate every **submask** of m — $O(3^n)$ overall

4.6 Digit DP

For "how many numbers in [L, R] satisfy P?" where R can be 10^{18}, iterate over *digits* rather than numbers. The state is (position, some property so far, `tight`), where `tight` records whether the prefix built so far exactly matches the bound — because if it does, the next digit is capped.

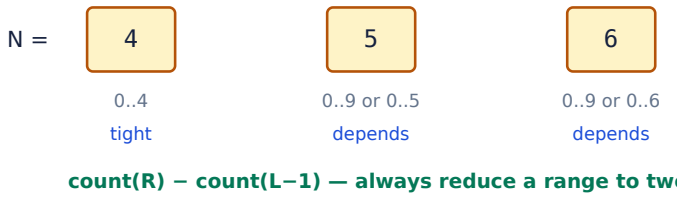

Fig 4.6 — bounding N = 456. While `tight` holds, the digit is capped by N's digit; once you place anything smaller, every later digit is free.

TWO STANDARD SIMPLIFICATIONS

Range to prefix: never handle [L, R] directly — compute `f(R) − f(L−1)`. **Leading zeros:** add a `started` flag when the property depends on the number's real length ("no two adjacent equal digits" must not be confused by padding zeros). With `tight` false and `started` true, the remaining count depends only on the position, so memoisation is effective — that is why the whole thing is O(digits × states × 10).

4.7 Interval DP

When the answer for a range depends on where you split it, the state is the range itself and the loop is over **increasing length**, so every shorter range is ready when needed. Matrix-chain multiplication, burst balloons, optimal BST and palindrome partitioning are all this shape.

```
// Matrix chain: dp[i][j] = min scalar multiplications for matrices i..j.
for (int len = 2; len <= n; ++len)              // LENGTH outermost
    for (int i = 0; i + len - 1 < n; ++i) {
        int j = i + len - 1;
        dp[i][j] = INT_MAX;
        for (int k = i; k < j; ++k)             // where to split
            dp[i][j] = min(dp[i][j],
                           dp[i][k] + dp[k+1][j] + p[i] * p[k+1] * p[j+1]);
    }                                           // O(n^3)
```

THINK BACKWARDS ON 'BURST BALLOONS'

The natural framing — “which balloon do I burst first?” — does not decompose, because bursting changes who the neighbours are for every later step. Invert it: ask **which balloon is burst last** in the range. That one's neighbours are the fixed range boundaries, so the left and right subranges become independent. Reversing the order of choice to make subproblems independent is a genuinely transferable interval-DP trick.

4.8 Interview questions

Q33 When is a problem a DP problem rather than greedy?

Greedy is valid when a **locally optimal choice is provably globally optimal** — formally, when the problem has the greedy-choice property and a matroid structure lurking underneath. DP is needed when a choice that looks worse now can pay off later, so you must keep every viable partial answer.

The practical test in an interview: **try to break greedy with a small counterexample**. Coin change with {1, 3, 4} and amount 6: greedy takes 4 + 1 + 1 = three coins, optimum is 3 + 3 = two. That single example proves greedy is wrong here and justifies DP in one sentence.

Conversely, greedy *is* provably right for: activity selection by earliest finish time, Huffman coding, fractional knapsack, Dijkstra, and MST. Notice 0/1 knapsack needs DP while *fractional* knapsack is greedy — the indivisibility is exactly what breaks the exchange argument.

Q34 Explain the difference between memoisation and tabulation. When is each better?

Memoisation is top-down: write the natural recursion, cache results. **Tabulation** is bottom-up: fill a table in dependency order, no recursion.

Memoisation wins when the state space is large but *sparse* — it only computes states actually reachable. It is also much closer to the brute force, so it is faster and safer to write under time pressure, and the recurrence is more readable.

Tabulation wins on constant factors (no call overhead, no hash lookups), avoids stack-overflow risk on deep recursion, and — the big one — makes **space optimisation** possible: once you see the table filled row by row, you can drop to one or two rows.

Concretely: 0/1 knapsack tabulated collapses from O(nW) to O(W). You cannot do that with memoisation, because the recursion may ask for any row at any time.

Q35 Reduce 0/1 knapsack from O(nW) space to O(W). Why does the loop direction matter?

Row i depends only on row $i-1$, so one array suffices. Iterate capacity **downward**:

```
for (int i = 1; i <= n; ++i)
    for (int w = W; w >= wt[i]; --w)
        dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
```

Going downward, `dp[w - wt[i]]` has not yet been touched in this iteration of i, so it still holds row $i-1$ — item i is therefore considered at most once. Going upward, that cell may already include item i, so the item gets reused, which silently solves **unbounded** knapsack instead.

Both directions are correct code for *some* problem, which is what makes this a good interview question: the bug produces plausible wrong answers rather than a crash. A useful sanity check is a single item with weight 1 — forward iteration will happily take it W times.

Q36 Edit distance: derive the recurrence and state the complexity.

`dp[i][j]` = minimum operations to turn the first i characters of A into the first j of B.

- If `A[i-1] == B[j-1]`: `dp[i][j] = dp[i-1][j-1]` — free.
- Otherwise `1 + min(dp[i-1][j-1]` (replace)`, dp[i-1][j]` (delete)`, dp[i][j-1]` (insert)`)`.

Base cases `dp[i][0] = i` and `dp[0][j] = j` — deleting or inserting everything. Time O(mn), space O(mn), reducible to **O(min(m,n))** by keeping two rows.

Follow-ups worth pre-empting: to **reconstruct** the operations you need the full table (or Hirschberg's divide-and-conquer, which recovers the alignment in O(mn) time and O(n) space); with **only insert and delete** allowed, the answer is m + n − 2×LCS; and if you only need to know whether the distance is ≤ k, band the DP to a diagonal strip for **O(k·min(m,n))**.

Q37 Given n ≤ 20 items, partition into two sets with minimum difference.

`n ≤ 20` is the signal: $2^{20} \approx 10^6$ subsets is directly enumerable. Iterate every mask, sum the selected items, and track `|total − 2×subset|`. $O(2^n \cdot n)$, or $O(2^n)$ if you compute sums incrementally with `sum[mask] = sum[mask & (mask-1)] + a[lowbit]`.

Two better answers depending on the follow-up:

- **Subset-sum DP** — a `bitset<MAXSUM>` of achievable sums, shifted by each element: `reachable |= reachable << a[i]`. O(n · sum / 64), which is excellent when the values are small even if n is large.
- **Meet in the middle** — if n reaches 40, split into halves, enumerate 2^{20} sums each, sort one and binary-search the other. O(2^(n/2) · n). That is §6.4.

Naming which technique the constraint is pointing at is the actual test here.

Q38 What is the difference between longest common subsequence and longest common substring?

A **subsequence** may skip characters; a **substring** must be contiguous. The DP tables differ in one line, and the difference is instructive.

LCS: `dp[i][j] = dp[i-1][j-1] + 1` on a match, else `max(dp[i-1][j], dp[i][j-1])`. The `max` is what allows skipping. The answer is `dp[m][n]`.

Longest common substring: `dp[i][j] = dp[i-1][j-1] + 1` on a match, else **0** — a mismatch breaks the run entirely. The answer is the maximum over the whole table, not the last cell.

Both are O(mn). For substrings specifically, a suffix automaton or suffix array gives O(m + n), and hashing plus binary search on the length gives O((m+n) log). LCS has no subquadratic algorithm under SETH — that is a proven conditional lower bound, and mentioning it is a strong signal.

Q39 How do you handle DP where the state includes 'at most k transactions'?

Add k as a dimension. For the stock-trading family: `dp[t][i]` = best profit using at most t transactions through day i. The naive recurrence scans all previous buy days and is $O(k \cdot n^2)$; the standard optimisation keeps a running `best = max(best, dp[t-1][j] - price[j])` so each state is O(1), giving **O(kn)**.

```
for (int t = 1; t <= k; ++t) {
    int best = -prices[0];                      // max over j of dp[t-1][j] - price[j]
    for (int i = 1; i < n; ++i) {
        dp[t][i] = max(dp[t][i-1], prices[i] + best);
        best     = max(best, dp[t-1][i] - prices[i]);
    }
}
```

The key edge case: if `k ≥ n/2` the constraint is not binding — you can take every profitable move — so fall back to the greedy “sum every positive difference” in O(n). Without that check, a problem with $k = 10^9$ allocates a table that does not fit. Interviewers plant exactly that input.

Q40 Explain why the coin-change loop order changes the answer.

Because the two nestings count different objects.

Coins outer, amount inner — each coin is offered to the whole amount range once, so a given multiset of coins is built in exactly one order. This counts **combinations**: {1,2} and {2,1} are the same solution, counted once.

Amount outer, coins inner — at every amount you may add any coin, so the same multiset is reachable by several orderings. This counts **permutations** (ordered sequences), which is what "Combination Sum IV" actually wants despite its name.

For the *minimum number of coins* variant the order does not matter, because `min` is commutative and idempotent — only counting is sensitive. That asymmetry is exactly what the question is probing.

Q41 Design a DP for 'maximum sum with no two adjacent elements', then extend it to a circle.

Linear: `dp[i] = max(dp[i-1], dp[i-2] + a[i])` — either skip element i or take it and jump over its neighbour. Two scalars suffice, so O(n) time and O(1) space.

Circular: the first and last elements are now adjacent, so they cannot both be taken. Rather than inventing a new recurrence, **split into two linear problems**: the best over `a[0..n-2]` and the best over `a[1..n-1]`, then take the maximum. Each excludes one of the conflicting pair, and every valid solution is covered by at least one of the two runs.

Edge case: $n = 1$, where both slices are empty — return `a[0]` directly. The general lesson is worth stating: *a circular constraint usually reduces to two linear runs, each pinning one end.*

Q42 What makes a problem NOT amenable to DP?

Three failure modes, and naming them shows you understand the preconditions rather than pattern-matching.

- **No optimal substructure.** The longest *simple* path in a general graph: combining optimal subpaths can revisit a vertex, so the pieces do not compose. It is NP-hard, and no DP over vertices fixes that.
- **No overlapping subproblems.** Merge sort has perfect substructure but every subproblem is distinct, so caching buys nothing — it is divide and conquer, not DP.
- **The state is too large to represent.** If correctness requires remembering an arbitrary *set* and n is 10^5, there is no table. This is where you switch to greedy with a proof, an approximation, or a flow formulation.

Recognising the third case and saying "the state would have to be exponential, so I would look for a different formulation" is a much better answer than forcing a DP that cannot fit.

Q43 Explain the O(n log n) LIS algorithm to someone who only knows the O(n²) one.

The O(n²) version asks, for each element, “what is the best subsequence ending before me that I can extend?” — a linear scan backwards.

The insight is that you never need to know all of them. For each achievable *length*, only the **smallest possible tail** matters, because a smaller tail can be extended by strictly more future elements and is never worse. So keep one number per length, in an array `tails`.

That array is automatically **sorted** — a longer subsequence cannot have a smaller tail — which is what lets binary search replace the linear scan. Each element either extends the array (a new longest length) or overwrites the first tail ≥ itself (same length, now cheaper to extend). O(n log n).

Close by stating the caveat before being asked: the array's *length* is the answer, but its *contents* are not the LIS itself.

Q44 How would you count paths in a grid with obstacles, and what if the grid is 10⁹ × 10⁹?

Small grid: `dp[i][j] = dp[i-1][j] + dp[i][j-1]`, with obstacle cells set to 0. O(mn) time, O(n) space with a rolling row.

Huge grid, few obstacles: the DP is impossible — 10^{18} cells — so switch to combinatorics. With no obstacles the count is `C(m+n-2, m-1)`, computable in O(m+n) with modular inverses. With k obstacles, sort them and use **inclusion-exclusion**: `f(i) = paths to obstacle i − Σ_{j before i} f(j) × paths(j → i)`, giving **O(k²)** independent of the grid size.

The transferable point: when a dimension explodes but the number of *interesting features* stays small, stop iterating over the dimension and start iterating over the features.

PART 5

String Algorithms

Linear alternatives to the obvious quadratic scan — and the argument for why they are linear.

String algorithms are where the O(nm) brute force is so obvious that the interviewer is really asking whether you know the linear alternative *and why it is linear*. The answer is almost always the same idea: never re-examine a character you have already matched.

5.1 KMP and the prefix function

Naive matching restarts the pattern at every text position, so a mismatch throws away everything learned: O(nm). KMP precomputes, for every prefix of the pattern, the length of the **longest proper prefix that is also a suffix** — the LPS array. On a mismatch it slides the pattern by the amount that array licenses, and never moves the text pointer backwards. Total **O(n + m)**.

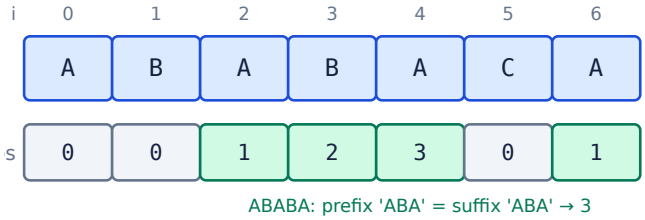

Fig 5.1 — LPS for `ABABACA`. At index 4 the prefix `ABA` also ends the window, so a mismatch there resumes at pattern position 3 rather than 0.

```
vector<int> build_lps(const string& p) {
    vector<int> lps(p.size(), 0);
    for (int i = 1, len = 0; i < (int)p.size(); ) {
        if (p[i] == p[len])      lps[i++] = ++len;
        else if (len)            len = lps[len - 1];   // fall back, do NOT reset to 0
        else                     lps[i++] = 0;
    }
    return lps;
}

int kmp(const string& text, const string& p) {
    auto lps = build_lps(p);
    for (int i = 0, j = 0; i < (int)text.size(); ) {
        if (text[i] == p[j]) { ++i; ++j; if (j == (int)p.size()) return i - j; }
        else if (j)          j = lps[j - 1];          // slide the pattern, i stays put
        else                 ++i;
    }
    return -1;
}
```

WHY IS IT LINEAR IF THERE IS A LOOP INSIDE A LOOP?

The same aggregate argument as the monotonic stack (§2.7). `i` never decreases, and `j` increases at most once per increment of `i` — so the total number of times `j` can *fall* is bounded by the number of times it rose, which is at most n. The inner loop spends a budget it can never refill. Being able to give this argument matters more than reproducing the code.

5.2 The Z-algorithm

`Z[i]` is the length of the longest substring starting at *i* that is also a prefix of the whole string. It is computed in O(n) by maintaining the rightmost **Z-box** [l, r] seen so far and reusing its already-known values instead of recomparing.

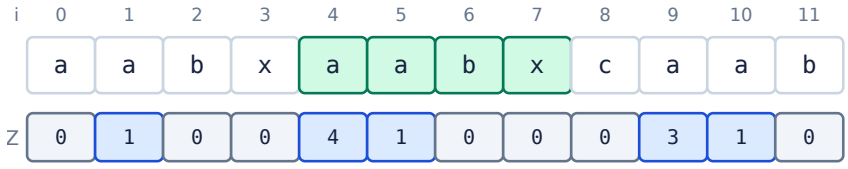

Fig 5.2 — Z-values for `aabxaabxcaab`. Z[4] = 4 because `aabx` reappears at index 4.

To use it for matching, build `P + '#' + T` where `#` occurs in neither, and look for `Z[i] == |P|`. Z is often preferred over KMP in practice: it is shorter, has no fall-back loop to get wrong, and the array itself answers more questions (longest common prefix with any suffix, string periodicity, and distinct substring counting).

5.3 Rabin-Karp and rolling hashes

Hash the pattern once and every window of the text, comparing hashes instead of characters. A **rolling** hash updates in O(1) per shift: `h = (h − s[i]·b^(m-1))·b + s[i+m]`. Average O(n + m); worst case O(nm) if every window collides.

Algorithm	Preprocess	Match	Worst case	Best for
Naive	—	O(nm)	O(nm)	tiny inputs, clarity
KMP	O(m)	O(n)	**O(n+m)**	single pattern, guaranteed bound
Z-algorithm	O(n+m)	O(n+m)	**O(n+m)**	same, simpler to write
Rabin-Karp	O(m)	O(n) avg	O(nm)	**multiple patterns**, 2-D search
Aho-Corasick	$O(\Sigma m_i)$	O(n + matches)	O(n + z)	many patterns at once
Suffix automaton	O(n)	O(m)	O(n+m)	many queries on one fixed text

ANTI-HASH TESTS ARE REAL

A fixed modulus and base can be reverse-engineered, and competitive judges ship tests that force collisions in single-mod hashing. Two defences: pick the base **randomly at runtime**, and use a modulus near 2^{61} with `__int128` arithmetic (or two independent 32-bit mods). With a random base and a large prime, the collision probability over *n* comparisons is about n·m/p — negligible. Saying "I would randomise the base to avoid adversarial collisions" is a strong signal.

5.4 Aho-Corasick

Searching *k* patterns one at a time costs O(k·n). Aho-Corasick builds a trie of all patterns, then adds **failure links** — the KMP fall-back generalised to a trie — so one pass over the text finds every occurrence of every pattern in **O(n + total pattern length + number of matches)**.

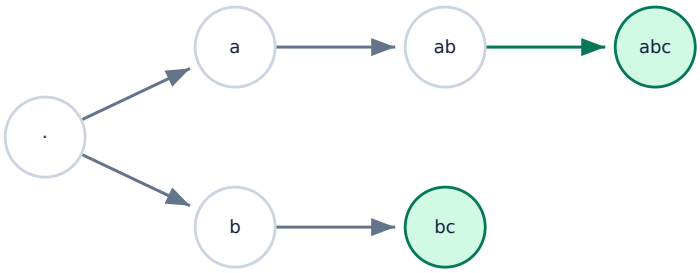

Fig 5.3 — a trie for `abc` and `bc`. The failure link from `ab` points at `b`, so a mismatch continues matching `bc` without rescanning the text.

This is what content filters, intrusion-detection signature engines and `grep -F` with many fixed strings actually use. The interview version is usually "find all words from a dictionary in a document" — if you name Aho-Corasick and explain that failure links are KMP's LPS lifted onto a trie, that is normally enough.

5.5 Interview questions

Q45 Find the longest palindromic substring. Give three approaches.

1. Expand around centres — $O(n^2)$ time, **O(1) space**. There are $2n-1$ centres (each character and each gap); expand outward while the characters match. This is the right answer for an interview: short, obviously correct, and easy to get right first time.

2. DP — `dp[i][j]` = is `s[i..j]` a palindrome, built by increasing length. $O(n^2)$ time *and* $O(n^2)$ space, so it is strictly worse than expansion unless you need every palindromic substring for a later step.

3. Manacher's algorithm — **O(n)**. Insert separators so every palindrome has odd length, then maintain the rightmost palindrome found and mirror its radii, exactly as the Z-algorithm reuses its Z-box. Mention it to show you know linear is achievable, but say plainly that you would write the expansion version unless the constraints demand otherwise.

Q46 Check whether two strings are anagrams, then group a list of words into anagram sets.

Two strings: compare frequency counts. For lowercase ASCII, a 26-int array is O(n) time and O(1) space. Sorting both and comparing is O(n log n) — correct but strictly worse, and interviewers notice the difference.

Grouping: the question becomes what to use as a hash key. Two options:

- **Sorted string** as the key — O(n · k log k) for n words of length k. Trivial to write.
- **Count signature** (a 26-length count encoded into a string) — O(n · k), better when words are long.

Avoid the "multiply primes assigned to each letter" trick unless you also raise the overflow problem yourself: the product exceeds 64 bits for words of moderate length, so it needs big integers or a modulus, and a modulus reintroduces collisions. Volunteering that caveat is worth more than the trick.

Q47 Explain the LPS array in KMP as if to someone who has never seen it.

Suppose you are matching `ABABC` against a text and you have already matched `ABAB` when the next character fails. The naive move is to shift the pattern by one and start over from the beginning. But you already know the last four text characters were `ABAB` — and `AB` is both a prefix and a suffix of that, so a fresh attempt is *already* two characters in.

The LPS array precomputes exactly that: for each prefix, the longest proper prefix that is also a suffix. On failure at pattern position *j*, jump to `lps[j-1]` instead of 0. Because the text pointer never rewinds, the whole scan is linear.

"Proper" matters — the whole string does not count, or the fallback would not make progress and the algorithm would loop forever.

Q48 Given a huge text and thousands of query words, find which appear.

Running KMP per word is O(k · n) — too slow when *k* is in the thousands.

Aho-Corasick: build one trie of all query words, add failure links, then make a single pass over the text. **$O(n + \Sigma|w_i| + \text{matches})$** — the text is read once regardless of how many patterns there are.

Two alternatives worth naming, since the best answer depends on which side is fixed:

- If the **text is fixed** and queries arrive over time, preprocess the text instead: a suffix automaton or suffix array answers each query in O(|w|) or O(|w| log n).
- If you only need *membership* and can tolerate false positives, a **Bloom filter** over the text's substrings is far smaller — useful at web scale.

Q49 How do you find the longest common prefix of an array of strings?

Vertical scanning is the practical answer: compare character 0 of every string, then character 1, and stop at the first mismatch or the end of the shortest string. O(S) where S is the total number of characters, and it exits early — if the first characters differ it is O(n).

Alternatives and when they matter: **horizontal** folding (LCP of the running prefix with each next string) is the same bound but does more work on average; **divide and conquer** is O(S) and parallelises; **binary search on the answer length** is O(S log m); a **trie** costs O(S) to build but then answers repeated queries and prefix questions cheaply.

Edge cases that catch people: the empty array, an empty string in the array (answer is always `""`), and a single-element array (the answer is that element).

Q50 Implement string matching where the pattern may contain '.' and '*'.

This is regular-expression matching — a 2-D DP, not a string algorithm. `dp[i][j]` = does `s[0..i)` match `p[0..j)`.

- `p[j-1]` is a normal character or `'.'`: `dp[i][j] = dp[i-1][j-1]` and the characters must match.
- `p[j-1] == '*'`: it applies to `p[j-2]`, so either **zero occurrences** (`dp[i][j-2]`) or **one more occurrence** (`dp[i-1][j]`, provided `p[j-2]` matches `s[i-1]`).

O(mn) time and space, reducible to O(n) with a rolling row. The base case that people miss: `dp[0][j]` is true for patterns like `a*b*c*` that can match the empty string, so it must be seeded rather than left false.

For the `'?'` / `'*'` wildcard variant, there is also a clean **greedy two-pointer** solution in O(m + n) that backtracks to the last `'*'` — worth mentioning as the follow-up.

Q51 Why might you use two different moduli in a rolling hash?

To make collisions vanishingly unlikely. With a single modulus p, two different substrings collide with probability roughly 1/p. Comparing all pairs among n substrings gives about $n^2/(2p)$ expected collisions — with $p \approx 10^9$ and $n = 10^5$, that is around 500 expected collisions. Not acceptable.

Hashing under **two independent moduli** and comparing the pair means a collision needs both to fail simultaneously: probability $\sim 1/(p_1 p_2) \approx 10^{-18}$. Effectively zero.

The alternative is a single modulus near 2^{61} using `__int128` intermediates, which gives comparable safety with one hash. Either way, **randomise the base at runtime** — a fixed base is what makes anti-hash tests possible in the first place.

Q52 Find the minimum window in S containing all characters of T.

Sliding window with two pointers. Keep a frequency map of what T requires and a counter of how many requirements are currently satisfied.

- Expand `right` until the window is valid.
- Then contract `left` while it stays valid, recording the best window at each step.
- Repeat to the end of S.

O(|S| + |T|). Each pointer only moves forward, so despite the nested loop the total work is linear — the same aggregate argument as §1.2 and §2.7.

The implementation detail that decides correctness: track a scalar `formed` counting how many *distinct* characters have reached their required count, and compare it against the number of distinct characters in T. Comparing whole maps on every step would add a factor of the alphabet size, and recomputing validity from scratch is the usual reason a candidate's version degrades to $O(n^2)$.

PART 6

Techniques & Patterns

The transformations that turn an intractable statement into a familiar one — and the constraint that signals each.

These are the techniques that turn an intractable-looking statement into a familiar one. Recognising which applies is most of the work; each is only a few lines once identified.

6.1 Binary search on the answer

The most under-used technique in interviews. If you cannot compute the answer directly but you *can* cheaply check "is x achievable?", and that check is **monotonic** — true for all x above some threshold and false below — then binary search the answer space.

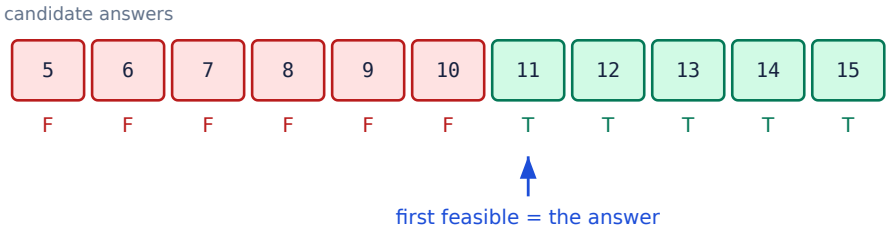

Fig 6.1 — the predicate must be monotonic: once feasible, always feasible. Binary search finds the boundary in O(log range) checks.

```cpp
// "Split the array into k parts, minimise the largest part sum."
// Direct construction is hard; CHECKING a candidate is trivial and greedy.
bool feasible(const vector<int>& a, int k, long long cap) {
    long long cur = 0; int parts = 1;
    for (int x : a) {
        if (x > cap) return false;
        if (cur + x > cap) { ++parts; cur = x; } else cur += x;
    }
    return parts <= k;
}

long long solve(const vector<int>& a, int k) {
    long long lo = *max_element(a.begin(), a.end());
    long long hi = accumulate(a.begin(), a.end(), 0LL);
    while (lo < hi) {                       // invariant: answer is in [lo, hi]
        long long mid = lo + (hi - lo) / 2;  // NOT (lo+hi)/2 — overflow
        if (feasible(a, k, mid)) hi = mid;   // mid works; maybe smaller does too
        else                     lo = mid + 1;
    }
    return lo;                              // O(n log(sum))
}
```

HOW TO SPOT IT

The giveaways are **“minimise the maximum”**, **“maximise the minimum”**, **“smallest k such that…”**, and any question about capacity, rate or threshold — ship-loading capacity, Koko eating bananas, aggressive cows, minimum days to make bouquets. When you see one, say: *“I cannot construct the optimum directly, but I can verify a candidate greedily, and feasibility is monotonic, so I will binary search the answer.”*

THE THREE CLASSIC BINARY-SEARCH BUGS

1. Overflow: `(lo + hi) / 2` overflows for large ints — use `lo + (hi - lo) / 2`. This was a real bug in the JDK's binary search for nine years.
2. Infinite loop: with `lo = mid` and integer division rounding down, `lo` can stop advancing. Either use `lo = mid + 1`, or round the midpoint *up* when assigning `lo = mid`.
3. Wrong boundary: decide up front whether you want the first true or the last false and keep one invariant throughout. Mixing the two mid-solution is the commonest cause of an off-by-one.

6.2 Two pointers and sliding windows

Pattern	Movement	Typical problem
Opposite ends	converge inward	two-sum on a sorted array, container with most water
Fast & slow	different speeds	cycle detection, find the middle of a list
Sliding window (fixed)	both move together	max sum of k consecutive
Sliding window (variable)	right expands, left contracts	longest substring without repeats
Merge-style	one per sequence	merge sorted arrays, intersection

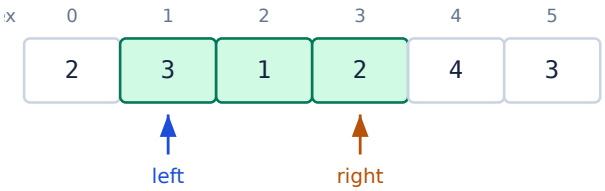

Fig 6.2 — smallest window summing to ≥ 7. The window [3,1,2] is contracted from the left while it stays valid; both pointers only move right, so the scan is O(n).

WHY IT IS O(n) DESPITE THE NESTED LOOP

The same aggregate argument that appears in §1.2, §2.7 and §5.1. `left` and `right` each advance at most n times across the entire run and never move backwards, so the total number of iterations is at most $2n$. If you can articulate this, you have understood the pattern rather than memorised its shape — and it is the single most reusable complexity argument in the book.

The precondition people forget: the opposite-ends variant needs the array *sorted*, and the variable-window variant needs the validity predicate to be *monotonic* in the window — extending a window must never make an invalid window valid. Sliding-window solutions for problems with negative numbers frequently fail exactly because that monotonicity does not hold; there, a prefix sum with a hash map is the correct tool.

6.3 Monotonic optimisation

Beyond the monotonic stack of §2.7, two named techniques appear in harder DP problems, and knowing their *names* and preconditions is usually enough:

- **Monotonic deque** — sliding-window maximum in O(n). Also optimises DP recurrences of the form `dp[i] = min(dp[j]) + c(i)` over a moving range of j, replacing an O(n) inner scan with O(1) amortised.
- **Convex hull trick** — when a transition looks like `dp[i] = min_j (m_j·x_i + b_j)`, each candidate is a *line* and the optimum is on the lower convex hull. That turns an $O(n^2)$ DP into O(n log n), or O(n) when slopes and queries are both monotonic.

6.4 Meet in the middle

When n is around 40 — too large for 2^n but suspiciously small — split the input in half, enumerate all 2^(n/2) possibilities on each side, and combine by sorting and binary searching. **O(2^(n/2) · n)** instead of $O(2^n)$: $2^{20} \approx 10^6$ rather than $2^{40} \approx 10^{12}$.

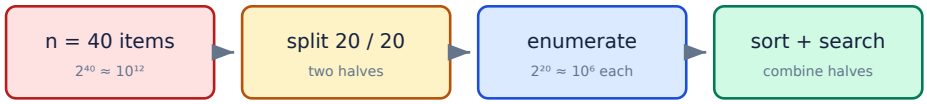

Fig 6.3 — a square-root reduction in the exponent. The constraint `n ≤ 40` is the tell.

6.5 Square-root decomposition

Split an array into √n blocks, precompute an aggregate per block, and answer a range query by touching O(√n) whole blocks plus O(√n) partial elements. Updates are O(1) or O(√n). It is strictly worse asymptotically than a segment tree, and frequently the better choice anyway: no recursion, contiguous memory, trivial to write correctly, and it handles operations a segment tree cannot compose.

Mo's algorithm is the offline variant: sort queries by (block of left endpoint, right endpoint) and move the window between consecutive queries. Total pointer movement is O((n + q)√n), which beats per-query recomputation for problems like “how many distinct values in this range” that have no easy associative merge.

6.6 Bit manipulation

Expression	Effect	Interview use
`x & (x - 1)`	clear the lowest set bit	count set bits; test power of two
`x & -x`	isolate the lowest set bit	Fenwick trees (§2.5)
`x | (x + 1)`	set the lowest clear bit	mask construction
`x ^ y`	differing bits	find the single non-duplicated element
`x & (x - 1) == 0`	is a power of two (x > 0)	capacity checks
`__builtin_popcount`	count set bits	subset sizes
`__builtin_clz`	leading zeros	floor(log2 x) in O(1)
`1 << k`	2^k	subset enumeration (§4.5)

SIGNED SHIFTS AND UB

`1 << 31` is **undefined behaviour** for a 32-bit signed `int` — write `1u << 31` or `1LL << 31`. Shifting by ≥ the width is also UB, so `1LL << 64` is not 0. And `__builtin_clz(0)` is undefined, not 32. These bite in exactly the edge cases interviewers test.

6.7 Interview questions

Q53 Find the single number in an array where every other element appears twice.

XOR everything. `x ^ x == 0` and `x ^ 0 == x`, and XOR is commutative and associative, so all the pairs annihilate regardless of order and the lone element survives. O(n) time, **O(1) space** — the hash-set solution is O(n) space and strictly worse.

The follow-ups are the real question:

- **Two singles, everything else twice:** XOR everything to get `a ^ b`. Take any set bit of that result — `d = z & -z` — and note it is a bit where a and b differ. Partition the array on that bit and XOR each half separately.
- **Every other element appears three times:** XOR no longer cancels. Count each bit position modulo 3, or track two accumulators `ones` / `twos` in a bitwise state machine.

Q54 You must minimise the maximum load across k machines. How do you approach it?

“Minimise the maximum” is the binary-search-on-the-answer signature. Constructing the optimal assignment directly is hard; *checking* a candidate is easy.

Binary search the candidate maximum load L over [max single task, sum of all tasks]. For each L, greedily fill machines: add tasks in order until the next would exceed L, then start a new machine. If you use at most k machines, L is feasible. Feasibility is monotonic — a larger L can only help — so the search is valid. **O(n log(sum))**.

Be precise about what the greedy check assumes: it works because tasks must stay in the given order (contiguous partition). If tasks may be assigned *arbitrarily*, the problem becomes bin packing, which is NP-hard — then longest-processing-time-first gives a 4/3-approximation. Distinguishing these two cases is exactly what the question is testing.

Q55 Detect a cycle in a linked list and find where it starts.

Floyd's tortoise and hare. Advance one pointer by one node and another by two. If they meet, there is a cycle; if the fast one reaches null, there is not. O(n) time, **O(1) space**.

Finding the entry point is the part worth deriving rather than memorising. Let the tail before the cycle have length a, and let the meeting point be b nodes into a cycle of length c. When they meet, slow has travelled $a + b$ and fast has travelled twice that, and fast's extra distance is a whole number of loops: 2(a+b) − (a+b) = a + b = kc. So a = kc − b — meaning walking a steps from the head and a steps from the meeting point arrives at the same node.

Hence: reset one pointer to the head, advance both one step at a time, and they meet at the cycle entrance. The hash-set alternative is O(n) space and gets no credit for the insight.

Q56 Given n = 40 items, count subsets summing to a target.

$2^{40} \approx 10^{12}$ is too many to enumerate, but `n = 40` is a deliberate hint: **meet in the middle**.

Split into two halves of 20. Enumerate all $2^{20} \approx 10^6$ subset sums of each half. Sort the second half's sums, then for each sum s in the first half, binary search for `target - s` and add the number of matches. **O(2^(n/2) · n)** ≈ 2×10^7 operations — comfortable.

If the *target* is small rather than n, the right answer is different: a subset-sum DP over a `bitset` is O(n · target / 64). Reading which dimension is bounded, and choosing the technique that exploits it, is the skill being tested.

Q57 Rotate an array by k positions in O(1) space.

Three reversals. Reverse the whole array, then reverse the first k elements, then reverse the remaining $n - k$. O(n) time, O(1) space.

```
void rotate(vector<int>& a, int k) {
    int n = a.size();
    k %= n; if (k < 0) k += n;                    // handle k > n and negative k
    reverse(a.begin(), a.end());
    reverse(a.begin(), a.begin() + k);
    reverse(a.begin() + k, a.end());
}
```

Why it works: reversing the whole array puts the last k elements at the front but in reverse order; the two local reversals repair the order within each block.

The alternative is **cyclic replacement**, moving each element directly to its final position — also O(n)/O(1), but it requires walking gcd(n, k) separate cycles, and getting that loop structure right under pressure is much harder. The reversal trick is three lines and obviously correct. The `k %= n` line is the edge case interviewers actually check.

Q58 How would you find the median of two sorted arrays in O(log(min(m,n)))?

Binary search the **partition**, not the values. Choose a cut point i in the smaller array; the cut in the larger is forced: `j = (m+n+1)/2 - i`, so that the combined left side always holds exactly half the elements.

The partition is correct when `A[i-1] ≤ B[j]` **and** `B[j-1] ≤ A[i]` — every element on the left is ≤ every element on the right. If `A[i-1] > B[j]`, i is too large, so search left; otherwise search right.

Binary searching only the **smaller** array gives O(log(min(m,n))). Use `INT_MIN` / `INT_MAX` sentinels for `i = 0` and `i = m` so the boundary comparisons need no special-casing — that is where almost every buggy implementation goes wrong.

Then the median is `max(A[i-1], B[j-1])` for odd total length, or the average of that and `min(A[i], B[j])` for even.

Q59 Explain how you would choose between a sliding window and a prefix sum.

Sliding window requires the validity predicate to be *monotonic* in the window: growing the window must never turn an invalid window valid. That holds when all values are non-negative and the condition is a sum or a count — then extending only increases the sum, so contracting from the left is a valid response.

Prefix sums with a hash map handle the cases where that breaks. With negative numbers, a longer window may have a *smaller* sum, so contracting is not justified and a window solution silently returns wrong answers. Instead compute running sums and, for "subarray sums to k", look up `prefix − k` in a map of previously seen prefixes — O(n) and correct regardless of sign.

Summary to give: *non-negative values and a monotonic condition → window, O(1) space; negatives or an exact-target count → prefix sums and a map, O(n) space.* Getting this distinction wrong is one of the most common silent bugs in interviews.

Q60 You have 10^9 integers on disk and 1 GB of RAM. Find the median.

The data does not fit, so the answer is about **passes over the data**, not about a clever in-memory algorithm.

Binary search on the value, counting per pass. The values are 32-bit, so the answer lies in a known range. Pick a midpoint, stream the whole file counting how many values fall below it, and recurse into the half containing the median. Each pass is O(n) I/O and halves the range, so 32 passes suffice — and only two counters are held in memory.

Better in practice: a **single-pass histogram**. Bucket by the high 16 bits — 65,536 counters, a few hundred KB. One pass identifies which bucket contains the median; a second pass histograms only that bucket's low bits. **Two passes**, trivial memory.

Also worth naming: external merge sort is the general fallback (O(n log n) I/O), and if an approximate answer is acceptable, t-digest or reservoir sampling gives a good median estimate in one pass and constant memory.

PART 7

The Interview Itself

The other half of the signal — how the same solution can pass or fail depending on how it is delivered.

A correct solution delivered badly loses to a slightly worse solution explained well, because the interviewer is predicting what you are like to work with. This part is about the other half of the signal.

7.1 A method for the 45 minutes

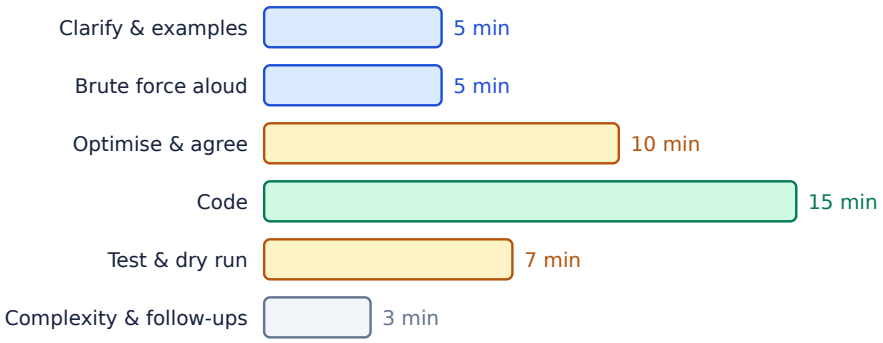

Fig 7.1 — a typical split. Note that coding is a third of it; candidates who start coding at minute two usually finish last.

Phase	What to do	What it signals
Clarify	restate the problem; ask about size, ranges, duplicates, empty input, and whether the input may be modified	you do not build the wrong thing
Example	work one small case by hand, on the board	you understand before you type
Brute force	state it and its complexity — do not code it	you always have something correct
Optimise	name the bottleneck, then the technique that removes it	you reason rather than recall
Agree	“shall I code this one?”	you do not waste 20 minutes on a rejected approach
Code	name variables properly; handle edges as you go	your production code is readable
Test	dry-run the example, then the edge cases you listed	you find your own bugs
Analyse	time and space, with the source of each term	you can predict behaviour at scale

THE SINGLE HIGHEST-VALUE HABIT

Think out loud, continuously. A silent candidate who produces the optimal answer at minute 40 scores worse than one who narrates a wrong idea at minute 10, spots the flaw, and corrects it. The interviewer is scoring your reasoning process, and silence gives them nothing to score. If you need quiet to think, say *“let me think about that for thirty seconds”* — that is itself communication.

FOUR AVOIDABLE FAILURES

1. Coding immediately. You will solve the wrong problem or paint yourself into a corner.
2. Silence while stuck. Say what you have tried and why it failed; interviewers hint readily but only to people who are visibly working.
3. Ignoring a hint. A hint is not a suggestion. If it is offered, engage with it immediately — it usually means your current path is a dead end.
4. Claiming a complexity you have not checked. Count the loops and the recursion before you name a bound; a confident wrong O() is worse than "let me work that out".

7.2 Ten questions that separate candidates

Q61 You said this is O(n log n). Where exactly does the log come from?

Point at the specific line. "The sort on line 3", "each heap push is O(log k) and there are n of them", "the recursion halves the range, so the depth is log n and each level is O(n)". A candidate who cannot localise the log usually recognised the shape of the problem rather than analysed their own code — and this question is designed to reveal exactly that.

Q62 Your solution works. Now the input does not fit in memory.

The whole answer is **passes over the data** and **what stays resident**. Streaming with bounded state (a heap of size k, a fixed histogram, a rolling counter); external merge sort when a total order is genuinely needed; chunking plus a merge phase; or an approximate structure — Bloom filter for membership, HyperLogLog for cardinality, t-digest or reservoir sampling for quantiles — if bounded error is acceptable. Ask whether it is, rather than assuming.

Q63 Why did you choose a hash map over a sorted array here?

Name the trade-off in both directions. Hash map: O(1) average lookup, no ordering, worse cache behaviour, higher constant memory. Sorted array: O(log n) lookup, but contiguous, supports range queries and predecessor/successor, and for small n is often faster in wall-clock time despite the worse bound. Then say which property of *this* problem decided it. "Because it is faster" is not an answer.

Q64 What breaks if the input has duplicates?

Have the answer ready before it is asked, because the interviewer chose the question knowing duplicates matter. Binary search must decide between first and last occurrence; two-pointer solutions need an explicit skip-duplicates loop or they emit repeated tuples; sorting is only safe if it is *stable* when ties carry meaning; and a set-based solution silently collapses multiplicities that the problem may care about.

Q65 Can you do it in O(1) space?

Usually one of four techniques. **In-place marking** — encode state in the sign or magnitude of the array itself. **Pointer manipulation** — reverse or relink instead of copying. **Bit tricks** — XOR to cancel pairs, a bitmask as a 32-element set. **Two pointers** instead of an auxiliary structure. Then state the cost honestly: in-place marking destroys the input, which may be unacceptable, and recursion is not O(1) space however elegant it looks.

Q66 How would you test this?

Give categories, not examples. **Empty and single-element** input; **boundaries** (0, 1, INT_MAX, negative); **duplicates and all-equal**; **already sorted and reverse sorted**; **the largest allowed input**, for performance; and **property-based / random testing against the brute force**, which is the strongest answer because it finds cases nobody thought to enumerate. Mentioning that you would keep the brute force specifically as a test oracle is a senior signal.

Q67 This is recursive. What if the depth is 10^6?

Stack overflow — typically at a depth of tens to hundreds of thousands, since the default stack is around 1–8 MB and each frame costs tens of bytes. Fixes, in order of preference: convert to iteration with an explicit stack; restructure to **tail recursion** if the compiler will eliminate it (not guaranteed in C++, so do not rely on it); use BFS instead of DFS when the traversal order is not important; or raise the stack limit, which is a deployment change rather than a fix. Recognising the risk unprompted is worth as much as the fix.

Q68 Your algorithm is correct. Now make it work with concurrent readers and writers.

Start by asking what consistency is actually required — that question is most of the answer. Then: a **reader-writer lock** when reads dominate; **fine-grained or striped locking** to reduce contention; **copy-on-write** when reads vastly outnumber writes and staleness is tolerable; **lock-free** structures with atomics and CAS when contention is extreme — while noting the ABA problem and the memory-reclamation difficulty. Raising **false sharing** (two hot variables on one cache line) shows real systems experience.

Q69 What is the worst-case input for your solution?

Construct it explicitly rather than describing it. Quicksort with a fixed pivot: already-sorted input. Hash map: keys engineered to collide. Two pointers: an input where the window never contracts. Union-Find without union by rank: a chain built in exactly the wrong order. Producing the adversarial case proves you understand which invariant your bound rests on — and it is the natural lead-in to how you would defend against it.

Q70 Would you ship this code?

An invitation to self-critique, and modesty here scores well. Name what you would change with more time: input validation at the boundary; overflow-safe arithmetic; extracting the magic numbers; a comment on the one non-obvious invariant; unit tests around the edge cases you listed; and a note on which assumption would break first at 100× the scale. Saying "yes, it is fine" about code written in fifteen minutes on a whiteboard reads as inexperience.

7.3 Complexity cheat sheet

Operation	Array	Sorted array	Linked list	BST (balanced)	Hash table	Heap
Access by index	**O(1)**	**O(1)**	O(n)	O(log n)*	—	O(1)
Search by value	O(n)	O(log n)	O(n)	O(log n)	**O(1)** avg	O(n)
Insert	O(n)	O(n)	**O(1)**	O(log n)	**O(1)** avg	O(log n)
Delete	O(n)	O(n)	**O(1)**†	O(log n)	**O(1)** avg	O(log n)
Find min/max	O(n)	**O(1)**	O(n)	O(log n)	O(n)	**O(1)**
Predecessor/successor	O(n)	O(log n)	O(n)	**O(log n)**	O(n)	O(n)
Ordered iteration	O(n log n)	**O(n)**	O(n log n)	**O(n)**	O(n log n)	O(n log n)

* with an order-statistic augmentation · † given a pointer to the node

Sort	Best	Average	Worst	Space	Stable	Notes
Quicksort	O(n log n)	O(n log n)	**O(n²)**	O(log n)	no	fastest in practice; `std::sort` is introsort
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	**yes**	`stable_sort`; the choice for linked lists
Heapsort	O(n log n)	O(n log n)	O(n log n)	**O(1)**	no	guaranteed bound, poor cache behaviour
Insertion sort	**O(n)**	O(n²)	O(n²)	O(1)	yes	excellent for n < 32 — used inside introsort
Counting sort	O(n+k)	O(n+k)	O(n+k)	O(k)	yes	small integer keys only
Radix sort	O(dn)	O(dn)	O(dn)	O(n+k)	yes	fixed-width keys; beats the comparison bound

7.4 Final checklist

- **Read the constraints first.** They name the intended complexity, and therefore the technique. `n ≤ 20` means bitmask; `n ≤ 40` means meet in the middle; `n ≤ 5000` permits O(n²); `n ≤ 10⁶` demands O(n) or O(n log n).
- **State the brute force before optimising.** It is your safety net and it makes the bottleneck concrete.
- **Name the bottleneck, then the technique.** "The inner scan is the O(n) factor; a monotonic stack removes it."

- **Confirm the approach before coding.** One sentence saves twenty minutes.
- **Handle edge cases in the code, not in a promise.** Empty, single element, all equal, overflow.
- **Dry-run your own code** on the example you worked out at the start.
- **Give complexity with its source** — which line, which term, and the space separately from the time.
- **Volunteer the trade-off you did not take,** and say when you would.
- **If you are stuck, say what you have ruled out.** That is progress, and it invites the hint.

THE LAST WORD

Every technique in this book is a way of not redoing work: memoisation caches it, two pointers refuse to revisit it, KMP remembers what it already matched, amortized analysis proves the redoing is bounded, and lazy propagation defers it until someone actually looks. If you internalise one idea, make it that one — when you are stuck, the productive question is almost always *“what am I computing more than once?”*